THE BOWSERBOY ENGINEERING BOOK

SECOND EDITION / NOVEMBER 2026

Building a Little Escape



A browser console, from pixels to publication

State. Shape. Light. Evidence.

Contents

Chapter links enter their opening pages. Stable IDs remain independent of pagination.

Before you pick up the console 3

Part 1 - The recoverable machine 6

CH01 - A console on a static website 7
CH02 - Runtime, cartridge and power 12
CH03 - Saves and recovery 17

Part 2 - The physical illusion 22

CH04 - Geometry and manufactured controls 23
CH05 - Pixels, lighting and fullscreen 30
CH06 - Input and interaction 35

Part 3 - Engineering the evidence 40

CH07 - Algorithms and data structures 41
CH08 - Design principles and trade-offs 47
CH09 - Testing and delivery 51
CH10 - Visual documentation and the development loop 57

Part 4 - Current lessons and transfer 63

CH11 - Sharpness without confusing the clocks 64
CH12 - Warm fabric and a genuinely green indicator 69
CH13 - Parallel tests and an exact-source release 75
CH14 - Change one appliance without losing its promises 80
Glossary and concept index 85
A maintainer's field guide 88
Source and claim register 91
Original chapter source register 94

Before you pick up the console

The opening question is deceptively small: can a Game Boy Advance emulator run on a GitHub static website? Yes, because a static host can deliver a program that executes in the reader's browser. The difficult work begins after that answer. A useful appliance must preserve progress, distinguish pause from power, deliver native input, recover from failed writes and show one coherent picture in several views. A convincing desk adds another system: geometry, lighting, camera fitting and performance all have their own clocks.

This book teaches that integration. It does not teach a new ARM CPU emulator implementation, include a commercial game collection or claim hardware-level compatibility for every cartridge. EmulatorJS and mGBA provide the native emulation. The independently authored shell makes those capabilities behave like a small physical object.

What this edition contains

Part I establishes ownership and durable state. Part II builds the visible and interactive appliance. Part III turns algorithms, design decisions, tests and documentation into a reasoning discipline. Part IV studies current sharpness, material, exposure and release problems, then asks you to design a safe change.

The first ten chapters preserve the existing Markdown teaching text, including its explicitly dated later notes. Their original implementation pin is `ab912d51ecc615f7aa55a4d27f6a876673739cdf`. Their account of former test counts, notice behavior, audio investigations and queued deployments is historical, not the current status. Each receives new second-edition practice. Chapters 11-14 and the references supply the current interpretation.

This is a new presentation and teaching extension, not a retroactive claim that the first edition had a PDF. The original source files and frozen public release package remain unchanged. The manifest hashes the exact working-tree inputs: a Git commit by itself cannot identify uncommitted documentation.

Three identities, three questions

Identity	What it establishes	What it does not establish
First public milestone, v1.0.0, November 6, 2026, Asia/Kolkata	Owner-designated first public availability; frozen package is based on qualified `c3d9dbd`	A later visual revision or a newly published GitHub Release
Qualified visual application `1b0385b`	Recorded successful Pages run `42135170801`, thirty matched public assets and four actual public cases	Qualification of every subsequent commit
Current source base `9ff921b`	Manual release workflow implemented and registered	A green current Pages run or an existing GitHub Release

At this edition's evidence cutoff, Pages run `42179075618` for `9ff921b` fails the 1440px reload-seating expectation: expected `seated`, observed `inserting`. Deployment is skipped. The first manual release attempt `42179152331` refuses to package that unqualified source. The timing cause is not established. This book does not repair it or convert its safety block into a successful release.

The startup-chime workstream also retains a narrower evidence gap: independent live proof of the specific chip-startup change was not located. That is not a proven audio defect, nor a listening-quality approval. Knowing the exact scope of a missing check is more useful than declaring everything broken or complete.

Reading routes

A newcomer should read in numbered order. A maintainer can begin with the glossary, revisit the relevant ownership chapter, then use the maintenance reference and current engineering chapters. Do not start a state-migration change with only the visual chapters, or a release decision with only a green local build.

Each chapter opener names its outcome and prerequisites. The teaching rhythm is picture, mechanism, failure, worked example, changed-condition retry and explained exercise. IDs such as `CH03-W01`, `CH03-R01`, `CH03-Q01` and `CH03-A01` survive pagination. Question and answer links go both ways. Original small exercises remain in the adopted chapters; these stable-ID exercises are the new second-edition learning layer.

Figures use labeled arrows. An arrow means the specific operation on its label, not implied trust or ownership of every downstream object. Most box-and-arrow figures run top to bottom. Images are evidence of an identified capture, not proof that the current checkout was deployed. The print version uses stills for animated sources; understanding a temporal claim still requires the relevant motion evidence.

Prerequisites and experiment boundaries

You need basic JavaScript functions, arrays, promises and browser events. Chapter 07 introduces the cost notation and projection units it needs. You do not need to know native emulator internals before reading Chapter 01. When the book reaches an upstream internal API, it names that boundary rather than pretending the API is stable.

Worked calculations use explicitly stated synthetic conditions. Unless a chapter cites a recorded execution, the calculation is a reasoning exercise, not a new benchmark. Browser experiments use the project's original Pixel Lab fixture, not the separately supplied game. No cartridge or firmware is included in the book package.

Public source links identify implementation and provenance. The source manifest identifies the bytes selected for this local edition. Neither is an exhaustive code-coverage guarantee or an independent rights audit. The appendix separates current guidance from frozen history and explains how to rebuild this reader without importing private context.

The promise to the reader

By the end, you should be able to predict what happens if two input sources overlap, a save transaction aborts, an acknowledgement arrives late, a paint is skipped, exposure changes or a release source has never passed its exact gate. You should also know which measurement would test your prediction. That is the book's purpose; its page count is not a measure of mastery.

PART 1

The recoverable machine

Begin with ownership, lifecycle and durable state.

1. A console on a static website
2. Runtime, cartridge and power
3. Saves and recovery

Picture the boundary. Explain the mechanism. Test the promise.

PART 1 / THE RECOVERABLE MACHINE

CH01

A console on a static website

> *Place execution, storage and trust at their real boundaries.*

Before reading
Browser files and basic JavaScript.

Contents

A console on a static website

> Adopted first-edition teaching text; dated later notes remain historical. New practice follows. Current status: orientation and Chapter 13.

Start with the ownership boundary

A static host serves bytes. It does not have to execute the emulator. The browser downloads the shell, loads the native core through EmulatorJS, reads a user-selected file and executes the game locally. The application has no ROM-upload endpoint and no cloud-save service.

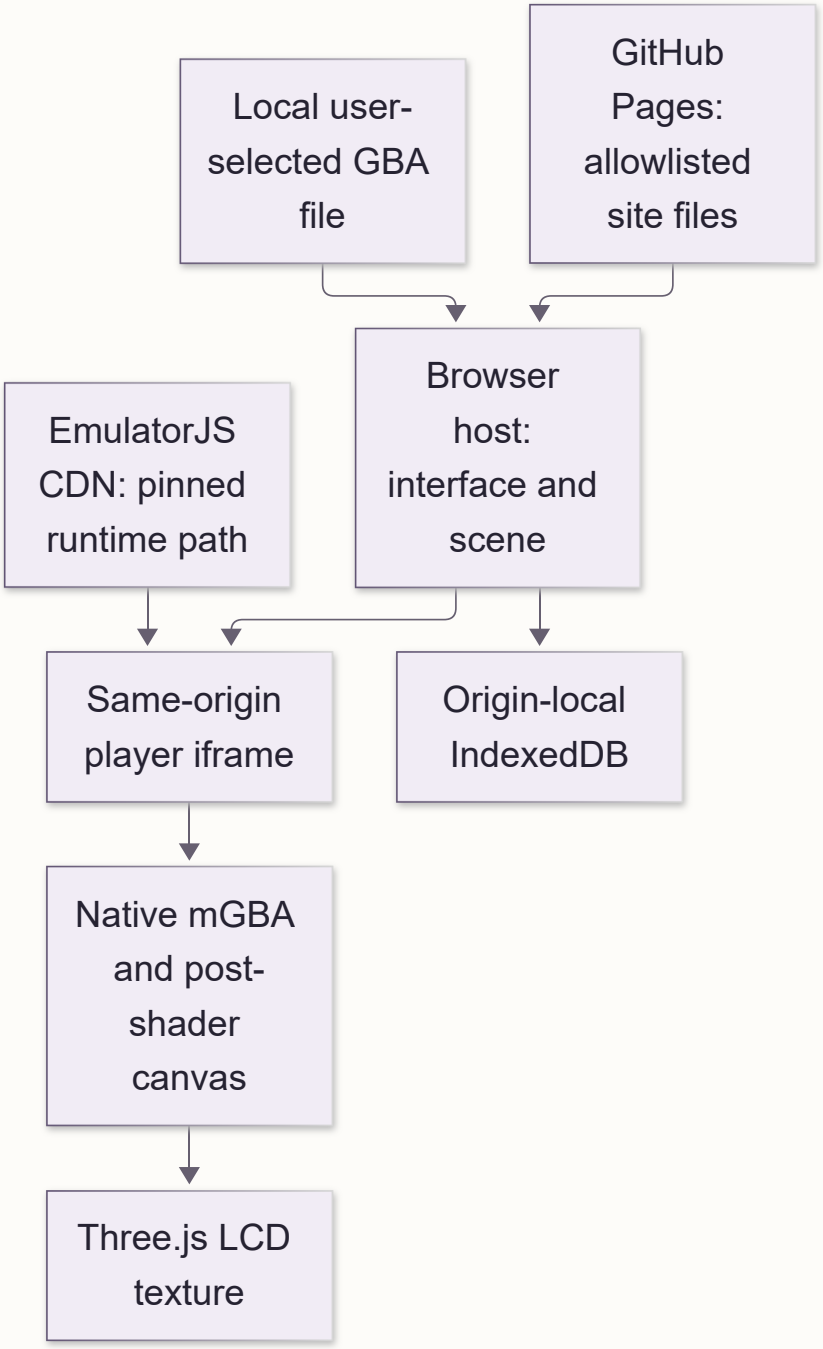


Figure 01-01. Arrows mean data or code delivery, not remote execution. Only the browser runs the game; CDN access is still a network dependency.

The host owns the player's intent: which cartridge, whether power is on, which save slot is selected and which physical view is active. The child owns the emulator instance. Three.js owns presentation, not CPU emulation. Keeping these responsibilities separate lets a change in desk lighting leave the game untouched.

Later website and console identity

Workstream47separates the umbrella website from its first console. BowserBOY uses the supplied chrome `assets/website-logo.png` in the HTML header and settings panels. BowserBOY ADVANCED is the user's name for the existing GBA console; its supplied purple `assets/logo.png`, LCD startup, rear engraving and bezel stay unchanged. Naming the future collection does not implement a console registry, additional cores, selection or save migration.

Keeping these assets separate is enough for the current requirement. Replacing the console loader's shared image would accidentally rebrand the handheld too; adding a generalized multi-console runtime would exceed the request. The owner retains responsibility for renaming the GitHub repository. Relative asset paths are preserved, but a future hosting-path change still requires actual deployment verification rather than a blanket compatibility promise.

Walk through the actual package

`siteFiles()` starts with named HTML/CSS and vendor entries, then selects first-level JavaScript and artwork from `src` and `assets`. `siteSource()` maps the public Three.js vendor names to installed package files. `build.mjs` copies this allowlist to `dist`, generates the original Pixel Lab cartridge and creates `.nojekyll`.

This is deliberately not "publish the repository." The engineering book, tests, development ledger and private continuity are not site payloads. A book link in the repository README is a GitHub documentation link, not a claim that the Markdown has been deployed as a new Pages reader.

All application paths are relative. A project site under `/BowserBoyAdvance/` therefore exercises a different condition from a page served only at `/`. The browser suite uses the project prefix so that a root-only success cannot conceal a broken Pages asset URL.

Static is not offline or untrusted-code isolation

The pinned CDN URL identifies a runtime release path; it is not an integrity hash of every returned byte. Runtime code runs with this site's origin privileges. The same-origin iframe provides lifecycle isolation, not a security sandbox against its own scripts. A statement that ROM bytes are not uploaded by this application must not become "the browser makes no network requests."

The single-threaded core avoids a requirement for custom cross-origin isolation headers. That is a compatibility choice, not a performance promise for every browser or phone. CDN failure, WebGL failure and unavailable browser storage remain real, separately reported failure modes.

Try it: distinguish the three claims

Exercise. The static build succeeds. Does this prove that the CDN is reachable, that a game starts, or that the public site has this build?

Answer. None of the three. Build success proves that the local packaging step completed. A real runtime test establishes startup in its tested environment. Exact public-asset comparison and live interaction establish the separate production boundary.

Source trail: S-01, S-02 and S-10; Architecture.

Second-edition practice: place the work

CH01-W01 - Trace one selected file

Suppose the browser receives three categories of bytes: the static shell from Pages, native runtime code from its pinned CDN path and a `.gba` file chosen locally. Draw three separate arrows. Only the first two are downloads in this model. The host checks the local file and gives it to a same-origin player; the browser executes the core. There is no application ROM-upload endpoint. This statement does not promise that arbitrary third-party code cannot make network requests.

CH01-R01 - Remove the network

Change one condition: the shell is already open, but required core files have not been fetched and the network disappears. Does static hosting make startup offline? No. A cached shell is not a cached native runtime. A sound offline design would need an explicitly licensed, complete, versioned cache strategy and failure UX; this application does not acquire that capability by its name.

CH01-Q01 - Is the iframe a trust boundary?

Explained answer

A same-origin runtime can be destroyed independently. Does that also prevent its scripts from accessing the host origin's storage?

CH01-A01 - Containment has a scope

Back to question

No. The iframe is a lifecycle boundary: teardown removes its native instance. Same-origin privileges remain. Hostile-code containment would require a different trust and resource-access design. Calling a frame a sandbox without that design confuses a useful integration technique with a security guarantee.

CH02

Runtime, cartridge and power

> *Predict cold boot, pause, restore and failed teardown.*

Before reading
Chapter 01.

Contents

Runtime, cartridge and power

> Adopted first-edition teaching text; dated later notes remain historical. New practice follows. Current status: orientation and Chapter 13.

Power is not a synonym for pause

Pause retains the running machine. Physical power off destroys it. A checkpoint restores an instant machine state. Battery data restores only the game's own save medium. These four operations cannot safely share one convenient handler.

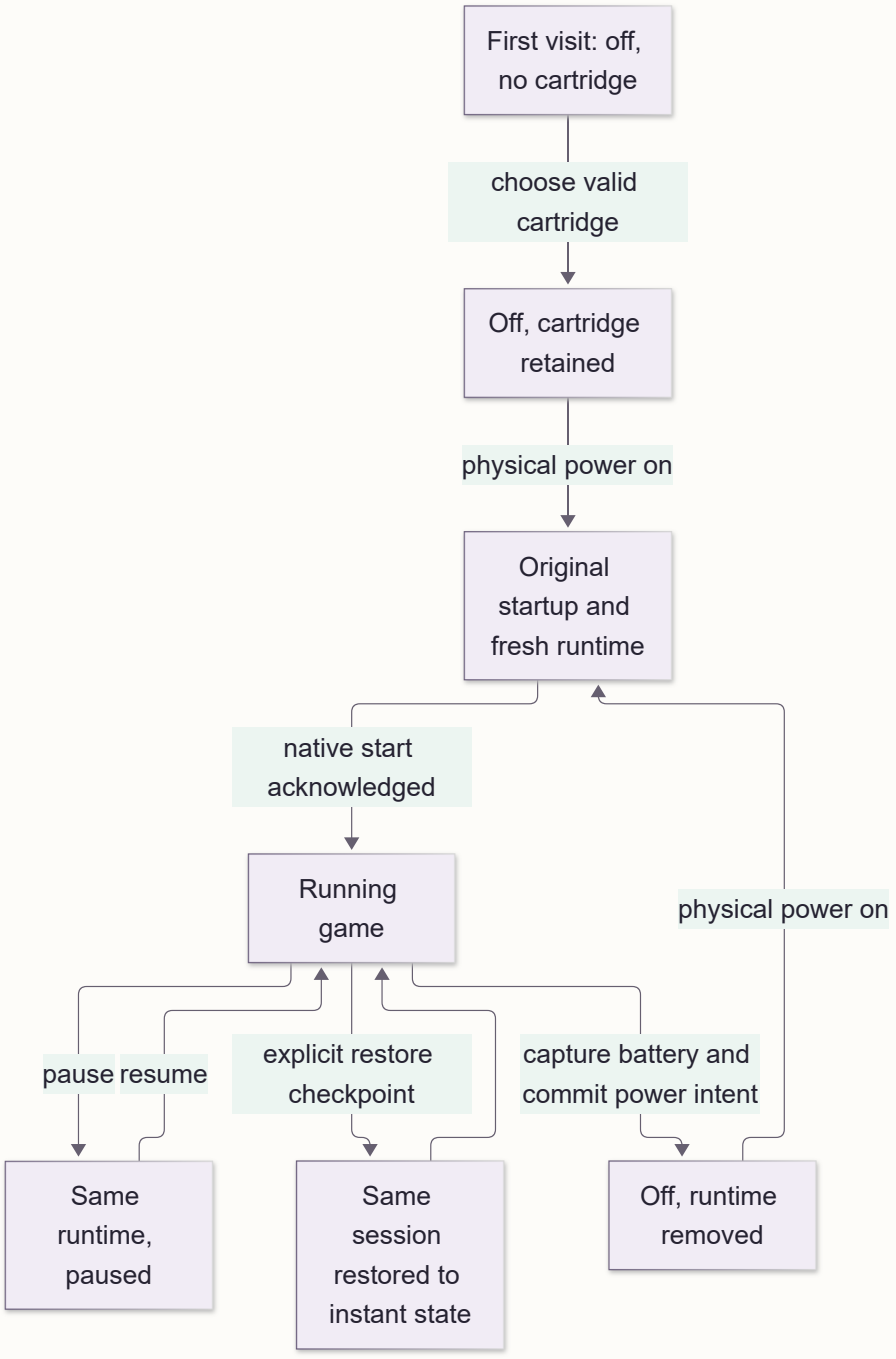


Figure 02-01. The power-on route is a cold boot with available battery data, not an automatic checkpoint load. Cartridge switching has its own recovery policy.

Cartridge selection is a transaction of intent

`loadRom()` identifies the proposed file before disturbing the current game. If another game is running, it checkpoints that game when automatic saving is enabled. With automatic saving disabled, it asks before discarding progress. The host reads the proposed game's record and rejects an incompatible runtime.

When storage is available, `rememberDevice()` records the new console intent before the host replaces its cartridge variables. An unavailable store is reported as a session-only limitation; a failed write is not falsely announced as remembered state.

If power is off, selection inserts the cartridge visually but does not launch the core. With power on, ordinary selection may resume the newest eligible checkpoint unless the record's `bootMode` is battery. This differs deliberately from pressing the physical power switch.

A fresh child contains the version-sensitive machinery

`launch()` clears the previous player and creates a same-origin iframe. It assigns a new random session ID, waits for the child's document and sends the file, identity, preferences and chosen restore data. The host has a bounded startup timeout and returns to a usable failure state if startup fails.

`initialize()` creates a temporary blob URL, configures the pinned native runtime and loads its script. On native startup it configures settings, revokes the ROM URL and either restores a checkpoint or loads battery bytes into a fresh core and restarts it. Its acknowledgement follows that work, not merely the iframe's HTML `load` event.

Both sides check origin, source window and session. Commands carry request IDs. A late acknowledgement from a previous cartridge must not resolve a new cartridge's request.

Failure order matters

During physical power off, battery-save capture and required storage writes happen before the host marks power off and clears the runtime. If the battery write fails, the operation reports that power remains on. Destroying the machine first would remove the user's chance to export the still-running session.

The original startup animation and synthesized chime belong to the shell. They are not a Nintendo BIOS and do not establish that the native core has already booted. Reduced-motion mode shortens the presentation independently of real loading.

Exercise. A player saves at level 3, makes a checkpoint at level 4, then cycles physical power. Which state should appear?

Answer. A fresh ROM boot using available in-game save data, not the level-4 checkpoint. The checkpoint remains separately available for explicit restore.

Source trail: S-01, S-02, S-04 and S-09; Saving.

Second-edition practice: preserve the escape route

CH02-W01 - A failed power-off write

Start with a running game and a newer in-game save still in native memory. The host captures the battery bytes, requests storage and receives a failed commit. The correct result is an explicit error with the runtime retained, not a black screen followed by a claim of success. The retained machine is the remaining route to export. The ordering is capture, required commit, then teardown; a successful capture alone cannot satisfy the commit step.

CH02-R01 - A late acknowledgement

Now replace the cartridge while an old child response is delayed. Request numbers alone are insufficient if they can repeat between sessions. Check the origin, sending window and session as well as request identity. The old response must not complete the new cartridge's command. A timeout ends waiting; it does not retroactively revoke a response already in transit.

CH02-Q01 - Which data belongs to cold boot?

Explained answer

A manual checkpoint is newer than battery data. Should physical power-on automatically choose the newer checkpoint?

CH02-A01 - Preserve operation meaning

Back to question

No. Physical power-on deliberately starts a fresh core with available battery data. Checkpoint restore is a separate operation. Choosing by recency here would erase the difference the interface promises, even though the result might initially look convenient.

PART 1 / THE RECOVERABLE MACHINE

CH03

Saves and recovery

> *Defend identity, integrity and the durable commit boundary.*

Before reading
Chapter 02 and promises.

Contents

Saves and recovery

> Adopted first-edition teaching text; dated later notes remain historical. New practice follows. Current status: orientation and Chapter 13.

Identify content, not a filename

`identifyRom()` checks an uncompressed `.gba` name, the 192-byte minimum, 32 MiB maximum and the fixed GBA header byte at offset `0xb2`. It then hashes the file's complete bytes with SHA-256. These checks are a useful format boundary, not a full proof that an arbitrary cartridge is valid or safe.

The hash is the storage key. Renaming a file therefore preserves its saves; patching even one byte produces a different identity. A title is presentation metadata, not a substitute for content identity.

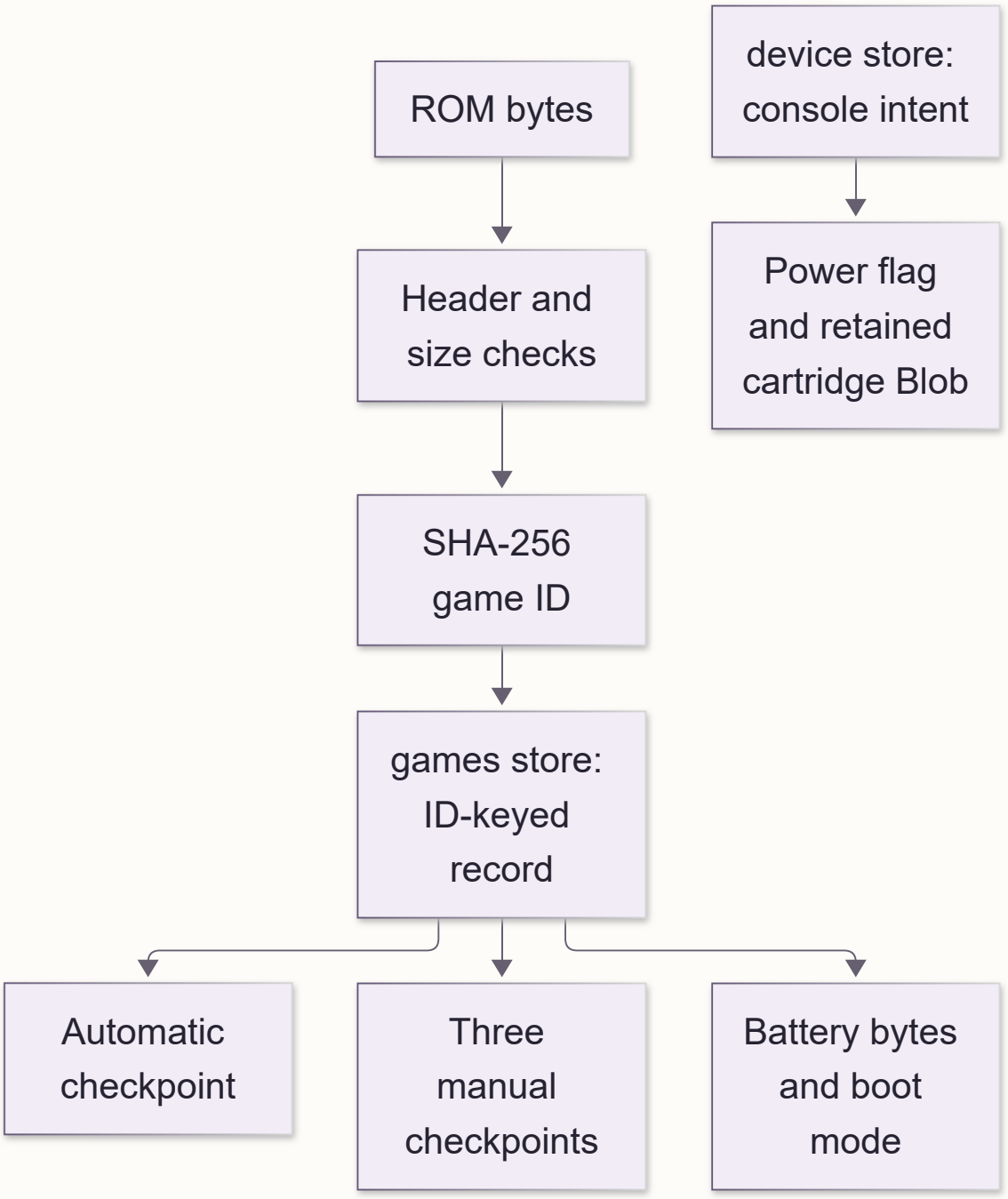


Figure 03-01. Game progress and remembered device intent have distinct records. A device refresh is not a new game identity.

There are three kinds of protection

An automatic checkpoint is a rolling convenience. Three manual slots provide independent rollback points. An exported `.bba-save` is portable recovery outside browser storage. None is a cloud synchronization service.

Each checkpoint includes native instant-state bytes and optional battery-save bytes. `putSlot()` also updates the record's battery field and selects checkpoint boot mode. `putBattery()` changes the battery field and boot mode without overwriting checkpoint slots. This is how a real power cycle can cold boot while preserving separately restorable snapshots.

The database has a `games` store keyed by game ID and a `device` store for the console record. An update reads and modifies a game within one read/write transaction. The promise resolves on transaction completion, not the first request callback. A successful `put` request followed by an aborted transaction is not a successful save.

Walk through a portable backup

`encodeBackup()` validates the capture, writes the format/version/runtime/ROM identity and encodes the binary arrays as Base64. It includes SHA-256 hashes of the state and optional battery bytes. `decodeBackup()` bounds file size, parses JSON, checks format and runtime, requires the exact ROM identity, validates Base64 and decoded capture sizes, then compares checksums.

The limits are deliberate: 16 MiB for native state, 1 MiB for battery bytes and 24 MiB for the JSON backup file. Base64 expands binary data by roughly one third, so encoded and decoded limits serve different purposes. Chunked conversion avoids expanding a large array into one function call.

Checksums detect damaged bytes; they are not signatures or encryption. An attacker who can rewrite a backup can also recompute its hashes. The runtime identifier is a compatibility fence, not a promise that future native versions can import every old snapshot.

Recover without success-shaped fallbacks

Storage is bound to the browser profile and origin, including the port. Switching from the public site to localhost is a different save location. Private browsing, eviction and clearing site data can remove records. The UI reports unavailable storage and keeps export useful instead of claiming a session was saved permanently.

The database-opening timeout and blocked/version-change handling avoid waiting forever behind another tab. They do not create backup copies elsewhere.

Exercise. A renamed ROM has the same bytes; a patched ROM has the same title. Which can import the original backup?

Answer. The renamed byte-identical ROM. Title equality does not bypass the exact content identity check. Export before migration; do not edit the backup's ROM field to disguise incompatibility.

Source trail: S-03 and S-04; recovery procedures.

Second-edition practice: distinguish integrity and identity

CH03-W01 - Budget the portable backup

For a synthetic state of 12 MiB and battery data of 0.5 MiB, Base64 needs about four thirds of the binary size: approximately 16.67 MiB before JSON metadata. That is below the 24 MiB file limit, while each decoded component also fits its own limit. These are three checks, not a promise that every JSON overhead fits an arbitrarily tight remainder. The actual decoder enforces exact limits.

CH03-R01 - Corruption with an intact title

Change a state byte but preserve the game's displayed title. The checksum must fail. If the ROM itself changes, content identity must fail independently. Titles are for people, checksums detect changed data and the ROM digest selects the game record. None substitutes for the others.

CH03-Q01 - Is request success transaction success?

Explained answer

An IndexedDB `put` request succeeds, then the containing transaction aborts. Can the interface announce that progress is durably saved?

CH03-A01 - Observe the commit boundary

Back to question

No. Wait for transaction completion. The request event describes one operation inside a transaction, not the final durable outcome. A synthetic test that emits request success followed by abort is valuable precisely because it distinguishes these two events.

PART 2

The physical illusion

Connect real geometry, one picture and overlapping input.

1. Geometry and manufactured controls
2. Pixels, lighting and fullscreen
3. Input and interaction

Picture the boundary. Explain the mechanism. Test the promise.

PART 2 / THE PHYSICAL ILLUSION

CH04

Geometry and manufactured controls

> *Distinguish shape, material and projected interaction.*

Before reading
Chapter 01; basic coordinates.

Contents

Geometry and manufactured controls

> Adopted first-edition teaching text; dated later notes remain historical. New practice follows. Current status: orientation and Chapter 13.

Shape must exist before shading can describe it

The handheld is an original hierarchy of Three.js meshes. The casing, shoulders, apertures, caps, cartridge, lower-edge switch and LCD are spatial objects, not a single screenshot decorated with CSS shadows. They can be inspected from the back and side without revealing a flat front-only illusion.

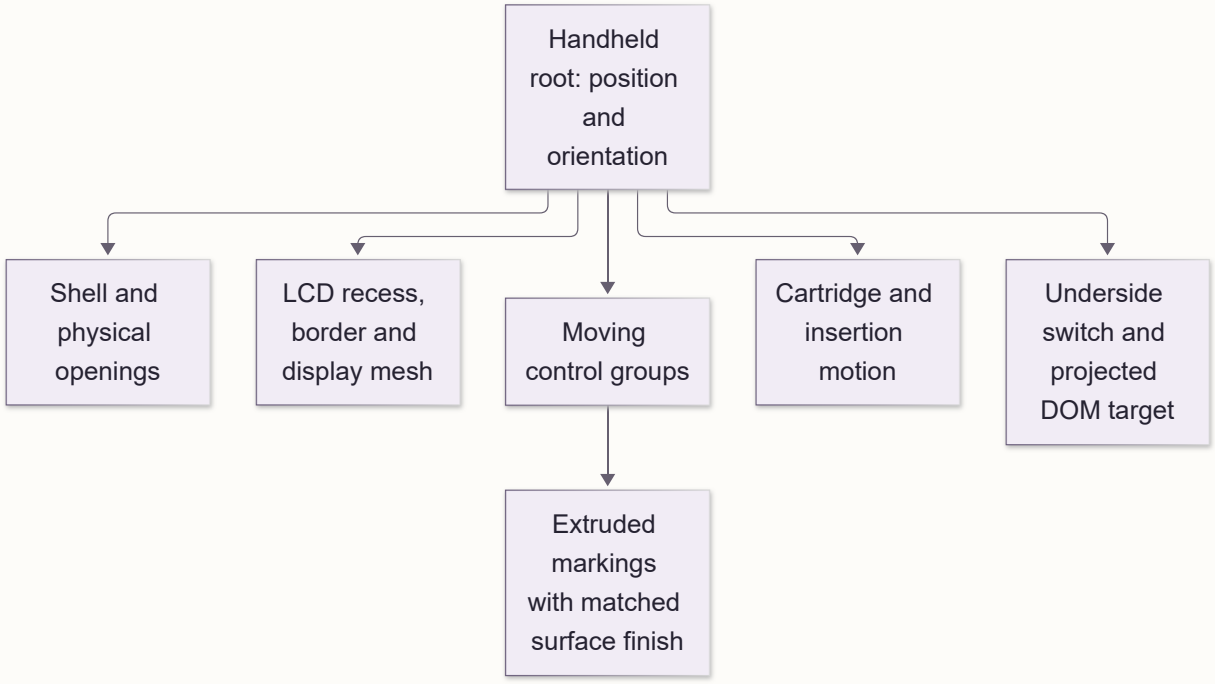


Figure 04-01. The hierarchy carries motion and material ownership. A control's marking moves with its cap rather than floating over the casing.

Why relief is different from printed lettering

A texture changes surface color. Relief changes surface shape and therefore silhouette, normals, highlights and self-occlusion. The shared polygon glyphs are extruded with bevels; counters preserve holes inside letters. The pinned source edition uses the cap's material for A/B/L/R markings and arrows; START/SELECT uses casing material. This lets light reveal a manufactured mark instead of a separate white decal.

The later contrast refinement in workstream 32 restores the earlier dark A/B and arrow tones with tinted clones of the rubber material. The extrusion, bevels, shadows and shared cap motion remain; only those six markings change color. L/R and START/SELECT retain the original material relationship.

Workstream 36 supersedes the raised A/B letters only: the user wants dark inward grooves. Their original polygon outlines cut the cap faces, while counter polygons remain solid islands. Dark floors and sidewalls sit below the original dome and move with the cap. Preserve the lathe's original rim normals and blend the new face normals into them; truncating the profile without doing this makes the center look like a separate disk. Raycasts must hit the inset floor before any cap surface, and counter probes must still hit cap height. Merely moving a dark letter below an uncut cap cannot pass that test. D-pad, shoulder, shell and maker markings retain their prior relief.

Workstream 38 then softens **only** the A/B groove material to `#999d98`. The original >3 contrast requirement is superseded by moderate 1.5-2.5 contrast against unchanged caps; actual contrast is 1.92740. Geometry, counters, shadows, other markings and the warm lamp remain unchanged. A shade refinement must not quietly turn a real groove back into paint or a raised letter.

Workstream 42 addresses a different meaning of "softer": the letter contours, not their shade. Dedicated quadratic/cubic action glyphs round A's joins and B's bowls, with broader .18x.19 proportions that thicken A's strokes. Leave the shared polygon font unchanged because the same A also belongs to shell labels. The cap openings, floor and walls use the same sampled contours, and counters remain cap-height islands. Test actual floor width and a rounded B corner next to its straight stem; mesh names alone cannot prove either visual property. Existing depth, contrast, lighting and press requirements remain in force.

The user then rejects that approximation in workstream 43. The earlier soft lettering was actually `bold 95px Arial`, not the shared polygon font or a new rounded alphabet. Reuse its original .25x.28 label footprint and -.005 vertical placement. March the antialiased mask's half-alpha boundary, stitch its closed contours, and distinguish the outer contour from counter islands. The cut, floor and walls share those recovered contours; this remains geometry, not a texture printed over an uncut cap.

Deep walls still harden the apparent font through shadow and occlusion. Make only those engravings shallow, with floors at local z=.047 and actual depth between .004 and .012. The earlier deep-groove bounds and bespoke 42 width/ corner tests are intentionally superseded, not silently removed: real inward rays/counters remain, and the actual triangulated floor must overlap an independently rendered original glyph by more than 94%. Preserve the shared font and all other controls. Browser/system font selection is still an original platform dependency; contour recovery does not license or bundle Arial.

The top Zanark mark and lower bezel serve different visual roles. Molded manufacturer lettering belongs to the shell. The bezel uses a star-free derivative of the supplied wordmark. Saturated orbital accents are removed in the derived texture; the original supplied PNG remains unchanged.

Fullscreen uses that same bezel texture rather than substituting a generic font. The first edition's neutral ink could look gold under the warm lamp; the later metallic chroma finish deliberately adds source color.

Workstreams 35 and 39 progressively reduce only that shared artwork: first to 75%, then to 60% of the unchanged canvas. The latter is another 20% reduction, centered with the source aspect intact. Tighten real-alpha inset checks and retain normal/fullscreen pixel identity; resizing the mesh or camera would solve a different problem.

Workstream 41 removes the surviving light exterior contour. A stricter color threshold damages the supplied letter gradients, so use solid alpha cores and flood-fill their original connected pixels instead. The seed filter only chooses components; it must not erode the reconstructed glyph edges or counters. The shared size, neutral shade and original purple artwork remain unchanged.

Workstream 45 supersedes only that neutral shade. Once the original alpha is reconstructed, source-in chrome bands and source-atop iridescent tint color only the letters. A lit material keeps the result part of the room: roughness .4, metalness .1, no emissive glow or new global environment. These are baked stylized reflections, not animated RGB or physically simulated thin-film color. The same canvas now lives in the decorative fullscreen DOM and supplies the Three.js texture. Reusing the bitmap avoids the one-channel rounding found when copying colored antialiased edges, while preserving exact RGBA equality. See the current source/evidence delta.

Workstream46treats the rear differently: enlarge its full supplied artwork from 2.1to2.95units and cut real shallow cavities into only the battery cover's rear face. Nested alpha contours preserve transparent counters as solid plastic islands. Colored floors and matching-plastic walls sit .006units inward; the outer bevel, thickness and grips are preserved. Keep the original1024x342RGBA and bright basic material on those floors, rather than dulling the logo to fake depth. Only the cavity mask uses512x171sampling. The same contour extractor retains the original A/B threshold/projection; no front-branding change follows. See the rear-logo source/evidence delta.



Figure 04-02. The public manufactured-control capture makes cap relief and bezel artwork inspectable. It is not evidence that the later notice revision was public when this image was recorded.

Coordinates become interaction targets

Workstream 37 strengthens physical feedback without changing those targets or delaying native input. Increase cap travel by about 60%, D-pad rocking and shoulder hinging, then use a more deliberate compression and quicker release. The mesh still approaches a bounded pose: no overshoot, cap scaling or detached lettering. Verify actual mapped-key displacement and equal-time compression/ release samples, not just renamed motion parameters. Recheck real pressed-cap illumination inside the existing openings; deeper travel must not recreate the socket-lid occlusion problem.

The existing adaptive-render contract still requires more than 99% compression within 100ms, even while GPU paints are skipped. The first slower response missed that bound. Correct the actual compression/release rates to 50/80, not the deadline or assertions; stronger feedback must not become sluggish input.

`project()` converts an object's world-space position into normalized device coordinates through the active camera, then into CSS pixels. This allows a physical switch to carry a usable DOM control while the console moves. Mesh shape and accessible hit target are related but not

identical: a tiny physical detail still needs the required touch area.

Normal play, insertion, inspection and tabletop have different envelopes. Insertion needs room above the slot; inspection allows deliberate rotation and zoom. The closer normal-play revision must not silently change these other poses or stretch the console to fill the screen.

The bounding-box trap

An axis-aligned box includes empty corners and hidden space. Projecting it can report a false collision even when the visible handheld is clear. Similarly, the transparent website logo's rectangle is larger than its visible lettering. The revised tests inspect actual visible geometry and actual logo alpha, while separately retaining switch-target and dock-clearance requirements.

That is not permission to weaken a collision assertion until a screenshot looks acceptable. It is choosing the physical quantity the requirement names.

Exercise. Why not make markings "3D" by adding a dark drop shadow?

Answer. The shadow would not change the mesh, material normals or inspected silhouette. It could also detach from a moving cap. Real relief ties appearance to geometry and shared motion; oblique inspection verifies that distinction.

Source trail: S-05, S-06 and S-11.

Second-edition practice: measure a manufactured feature

CH04-W01 - A letter with a counter

Imagine an A-shaped opaque mask with a triangular transparent counter. Its outer contour defines a cavity; the counter remains a solid plastic island. A floor below the cap and walls joining the lip make the letter an inward groove. A ray through the stroke must encounter the lower floor before an uncut cap; a ray through the counter must encounter cap height.

CH04-R01 - Change color, retain geometry

The user asks for softer gray lettering. Changing floor material can satisfy that request without altering contours or depth. If the request instead says rounder letterforms, material changes cannot satisfy it. Identify whether the controlled quantity is color, contour, depth or illumination before choosing the implementation and assertion.

CH04-Q01 - Why can a box test mislead?

Explained answer

An axis-aligned bounding box overlaps the toolbar, but the visible shell does not. What should the regression measure?

CH04-A01 - Match the requirement's geometry

Back to question

Use the relevant projected visible surfaces for the visual-clearance claim, and independently retain accessible hit-target bounds. The box is useful for broad-phase rejection, not conclusive visible collision. Replacing it requires evidence of the actual geometry, not simply weakening the threshold.

PART 2 / THE PHYSICAL ILLUSION

CH05

Pixels, lighting and fullscreen

> *Trace one native picture through presentation and light.*

Before reading
Chapters 02 and 04.

Contents

Pixels, lighting and fullscreen

> Adopted first-edition teaching text; dated later notes remain historical. New practice follows. Current status: orientation and Chapter 13.

One native picture, several presentations

The native core produces the game. Its shader processes that output. The shell consumes the retained post-shader canvas as a Three.js texture. It does not recreate CRT Geom with CSS or run a second emulator for fullscreen.

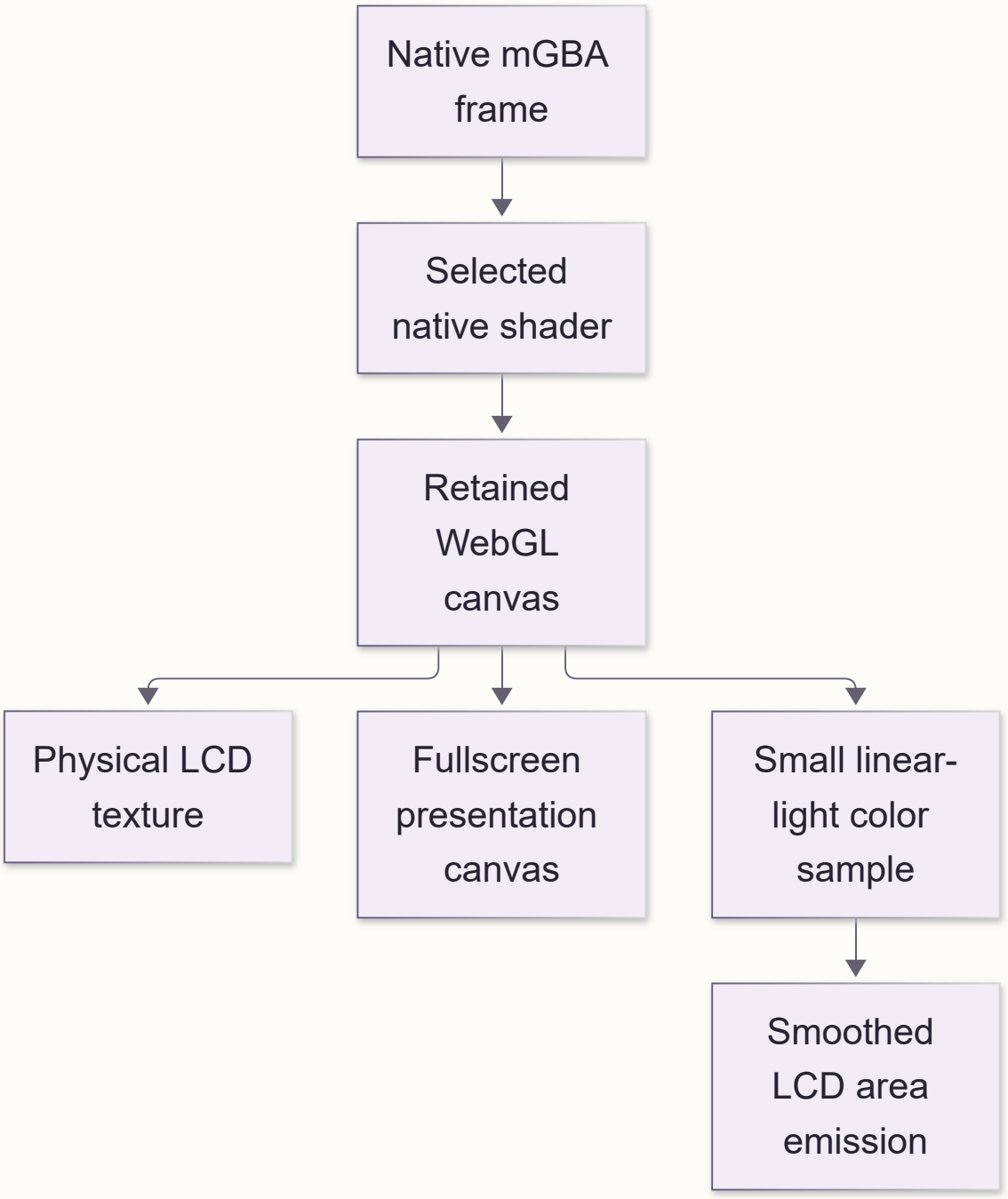


Figure 05-01. Presentation and illumination consume the same game output. The lighting sampler is not part of game simulation.

Retaining the framebuffer is a scoped compatibility choice

The adapter overrides the native canvas's WebGL context request to preserve its drawing buffer. Without retained output, the shell's later read can see an already-cleared frame. This behavior is confined to that core canvas, not every WebGL context on the page. It is version-sensitive integration, documented beside the pinned adapter.

The checked preference list exposes nine native CRT presets plus Unfiltered. CRT Geom is the default when no explicit choice exists. A valid saved preference is not overwritten merely because the default changed. Applying the filter does not require replacing the current cartridge.

Screen fullscreen draws the current picture inside a proportional decorative bezel and dark surrounding space. The native picture remains 3:2. The bottom-right 50-100% slider scales the presentation frame, not the emulated resolution, save state or CPU rate. The decorative wordmark uses the same derived image as the physical bezel. Browser refusal of fullscreen is an explicit error.

Lighting follows emitted content

`ScreenIllumination` samples a 24 by 16 image. It converts each sRGB channel through a 256-entry linear-light lookup table before averaging. Averaging display-encoded channel values directly would give a different, physically misleading result.

Sampling is bounded to a 125 ms cadence for live content, with an immediate sample when the source changes. The color converges using an exponential factor based on elapsed time. The peak channel determines intensity; normalized color preserves the tint. A black picture converges to no LCD emission. This is a low-cost whole-screen approximation, not per-pixel global illumination.

The paper lantern and room lights are independent scene systems. The lantern's spot, transmitted area light and finite tabletop bounce approximate a warm physical source. They are intentionally not a promise of unbiased path tracing.

Fabric refinement, November 6, 2026. The current shade uses a subtle neutral white weave in both color and emission maps. Mapping emission matters: an untextured luminous term can otherwise wash out the reflected fabric detail. A warmer cream emission changes the glow, not the clean fabric into brown stains. Actual rendered warmth, brightness and the existing no-seam bounds are checked without reducing renderer resolution.

Living implementation note, November 6, 2026. The first-edition lantern above is replaced by a clean STORSEGEL-style white drum shade and ash tripod. An exterior transmitted-light plane avoids a hard seam on the fabric; soft downward light through the open bottom reveals the legs. Lamp/room/LCD independence and full-resolution rendering remain intact. See the current lighting architecture for dimensions and source details; this note does not reissue the frozen book.

Presentation performance is not game speed

The scene can defer expensive painting under load and skip tabletop rendering while screen fullscreen is active. Those choices must not change the cartridge's native simulation clock. Conversely, setting the fast-forward flag is not sufficient if native VSync still caps the loop. The adapter explicitly reconciles that interaction; tests measure the original cartridge's clock.

Exercise. If the desk renderer drops to 20 painted frames per second, has the emulator necessarily slowed to one third speed?

Answer. No. Render cadence and native game progress are separate quantities. Measure cartridge progress over wall time rather than infer it from scene smoothness. Real system overload may affect both, but one is not proof of the other.

Source trail: S-01, S-02, S-06, S-07 and S-08.

Second-edition practice: separate a frame and its light

CH05-W01 - Average in the right space

Take two synthetic grayscale pixels, one black and one white. Their mean linear intensity is 0.5. Encoding that mean to sRGB gives approximately 0.735, not 0.5. Averaging already encoded channels would produce a darker energy estimate. The sampler converts channels to linear light before averaging; its 384-pixel reduction is a whole-screen approximation.

CH05-R01 - Keep the source, change the view

Enter screen fullscreen without changing the game. The same retained native canvas supplies the new presentation. A new ROM load would change lifecycle and risk progress for a display operation. A stretched 16:9 canvas would change aspect, not improve access to the original 3:2 picture.

CH05-Q01 - Can black emit a fixed fill?

Explained answer

Would adding a constant minimum LCD light improve dark-room usability without changing the light model?

CH05-A01 - Black is an invariant

Back to question

No. A constant fill would make a black screen illuminate its surroundings. The existing peak-dependent spill converges to zero at black. Improve controls through a deliberately separate interaction design if needed; do not silently change a content-driven emitter into an always-on lamp.

PART 2 / THE PHYSICAL ILLUSION

CH06

Input and interaction

> *Release only the source ownership that actually ended.*

Before reading
Chapter 02; browser input events.

CH06 / INPUT AND INTERACTION

Contents

Input and interaction

> Adopted first-edition teaching text; dated later notes remain historical. New practice follows. Current status: orientation and Chapter 13.

A button is the union of its input sources

Keyboard, pointer and other supported input routes can overlap. A single `pressed` boolean loses ownership: releasing one source would release a button still held by another source.

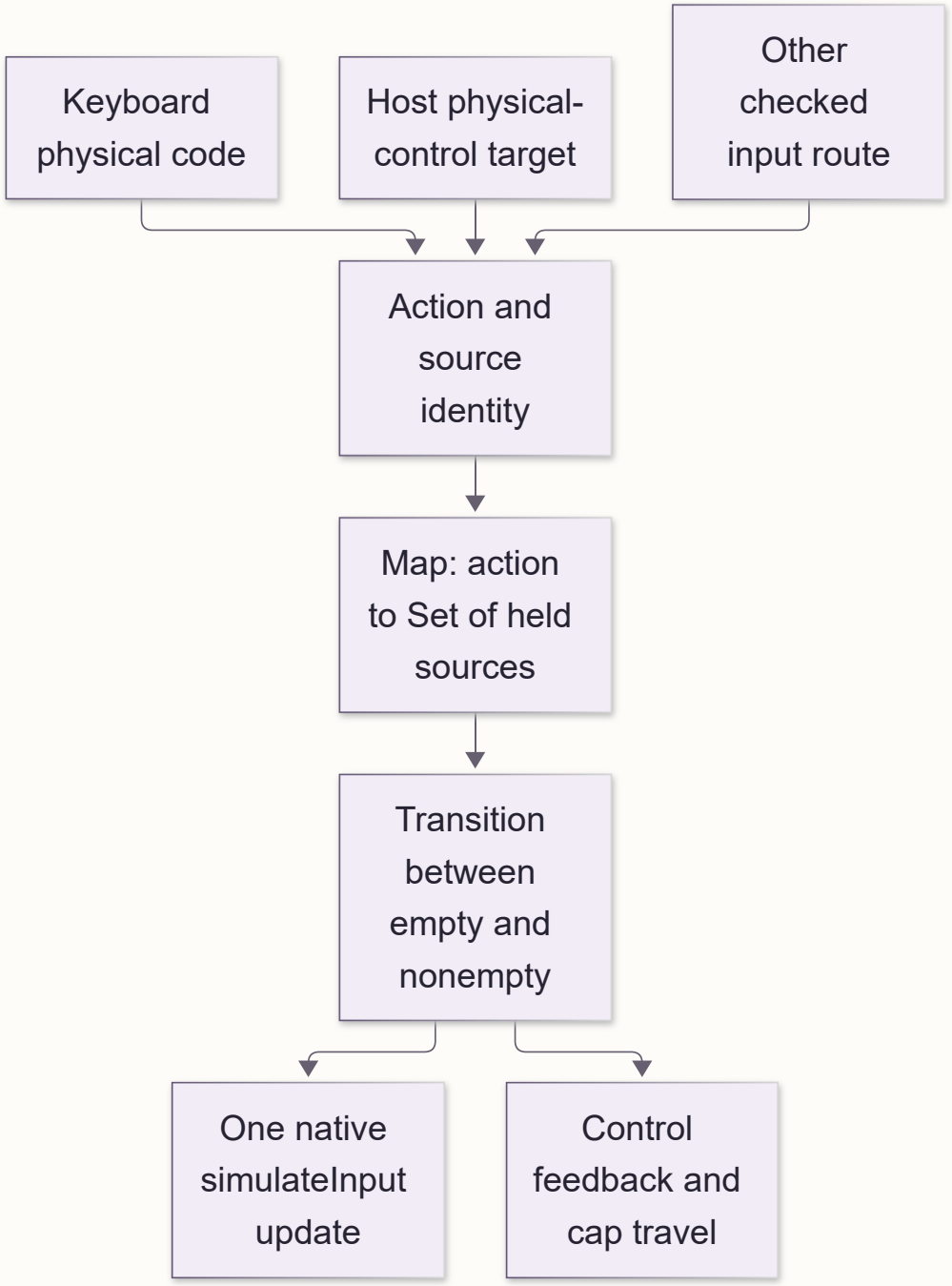


Figure 06-01. Native transitions occur when aggregate state changes, not on every duplicate press notification.

`input()` validates the action, boolean state and bounded source string. Each action maps to a `Set` of sources. Adding a source can change empty to nonempty; removing one can change nonempty to empty. Only those aggregate transitions call the native button API and notify the host.

For example, keyboard A and a pointer can both hold the same GBA action. Releasing the pointer leaves the keyboard source in the set. The native button stays down until that remaining source releases.

Remember the action that was pressed

`bindKeyboard()` stores physical key codes and their assigned actions in a `Map`. Keyup first checks that held map, even if the current mode would no longer accept a new keydown. This is important when focus or view changes between press and release.

Repeated keydown does not create repeated native presses. Blur releases all held keys. The child also releases keyboard holds when the document becomes hidden. Reconfiguration releases prior holds before replacing preferences, so a rebind cannot strand the old action.

The host and child coordinate input enablement around playing, inspection, transitions and native settings. Screen fullscreen deliberately retains keyboard gameplay even if entered from another desk pose. Escape is reserved for closing the active presentation or tools, not assigned as a game key.

Configuration is checked, not trusted

The fixed control table defines IDs, native button numbers, default physical codes and gamepad names. Preference validation rejects duplicate assignments and reserved or malformed keys. The set used during validation represents already-assigned codes; it is a different set from the runtime's held sources.

Advanced native settings are filtered so the shell retains ownership of keyboard, save handling and virtual-control presentation. This is a division of application responsibility, not proof that arbitrary third-party runtime options are harmless.

Visual feedback has a separate verification layer

A moving A cap proves visible response, not necessarily native input delivery. A native button press proves game input, not that the cap remained visible inside its aperture under lighting. The browser cases exercise both. Similarly, a physical switch can be tiny in world units while its projected DOM target still meets 44 CSS pixels.

Exercise. The user holds a key, opens settings and releases the key there. Why cannot keyup simply return whenever gameplay is disabled?

Answer. It would discard the release for an existing hold. Process held keyup and release ownership before rejecting new gameplay input. Mode changes also explicitly clear input so losing the event does not leave a stuck button.

Source trail: S-02, S-06, S-08 and S-11.

Second-edition practice: release only your ownership

CH06-W01 - Two sources hold one action

Start with `A -> {}`. Keyboard down gives `{keyboard:KeyX}` and one native press. Pointer down gives `{keyboard:KeyX, pointer:7}` without a second native press. Keyboard up leaves `{pointer:7}` without release. Pointer up leaves the empty set and produces one native release. This is a union of holds, not a counter blindly incremented on repeated browser keydown events.

CH06-R01 - Change mapping during the hold

Now rebind the physical key while it is held. Release prior ownership before applying the new mapping. If keyup were translated using only the new mapping, it could release a different action while stranding the old one. The held-key map remembers the action originally pressed.

CH06-Q01 - Where should a repeat go?

Explained answer

An operating system repeats keydown for a held key. Should every event create a new held source that needs another keyup?

CH06-A01 - Identity beats counting

Back to question

No. The physical source already exists in the set. Repeated down is idempotent for that hold. Counting repeated notifications would require releases that the operating system never sends. Set membership models the actual ownership invariant more directly than an event count.

PART 3

Engineering the evidence

Turn algorithms and failures into defensible decisions.

1. Algorithms and data structures
2. Design principles and trade-offs
3. Testing and delivery
4. Visual documentation and the development loop

Picture the boundary. Explain the mechanism. Test the promise.

PART 3 / ENGINEERING THE EVIDENCE

CH07

Algorithms and data structures

> *Reason about invariants, units and the actual bounded workload.*

Before reading
Chapters 03-06.

Contents

Algorithms and data structures

> Adopted first-edition teaching text; dated later notes remain historical. New practice follows. Current status: orientation and Chapter 13.

This chapter collects the algorithms the earlier chapters use. Costs below describe application-level work, not an unmeasured browser-engine guarantee.

7.1 Content addressing and bounded validation

For a ROM with `n` bytes, content hashing reads `O(n)` input and the current implementation materializes an `O(n)` byte array. A fixed-length digest becomes the record key. File-size checks happen before that allocation. The fixed header check is `O(1)`; it is not comprehensive ROM validation.

Backup decoding first bounds the JSON file, then the encoded strings and decoded arrays. The Base64 work and checksums are linear in save size. Chunking the encoder into 8192-byte pieces bounds the argument list passed to `String.fromCharCode`; it does not make the entire backup streaming or constant-memory.

`latestSlot()` builds a candidate array, removes empty entries, sorts descending ISO timestamps and selects the first. With `k` candidates, sorting is `O(k log k)`, but this application has only four candidates: automatic and three manual. A more complex priority queue would not improve this actual bounded use case. Equal timestamps do not mean "prefer a manual slot."

7.2 Maps and sets encode ownership

Structure	Key or element	Invariant
Game record	SHA-256 ROM ID	Filename changes do not change identity
Held-key Map	Physical code to action	Release uses the action actually pressed
Held-input Map	Action to source Set	Native pressed means at least one source remains
Assigned-key Set	Physical code	No two configured actions share a key
Pending command mapping	Request identity	A response belongs to its current request/session

These are not interchangeable collections. The held-source set models concurrent ownership; the assigned-key set enforces uniqueness at validation. Map/Set operations provide typical efficient lookup, while control-table searches scan only the fixed ten controls.

7.3 Perspective fitting with UI exclusions

Let `t = tan(verticalFov / 2)`, viewport height be `H` CSS pixels and camera distance be `d` world units. The vertical scale is approximately:

```
pixelsPerUnit = H / (2 * t * d)
requiredDistanceForWidth  = envelopeWidth  * H / (2 * t * availableWidth)
requiredDistanceForHeight = envelopeHeight * H / (2 * t * availableHeight)
```

Widths and heights of the envelope are world-space design bounds. Available region dimensions are CSS pixels. Confusing these units is a reliable way to produce a fit that works on only one viewport.

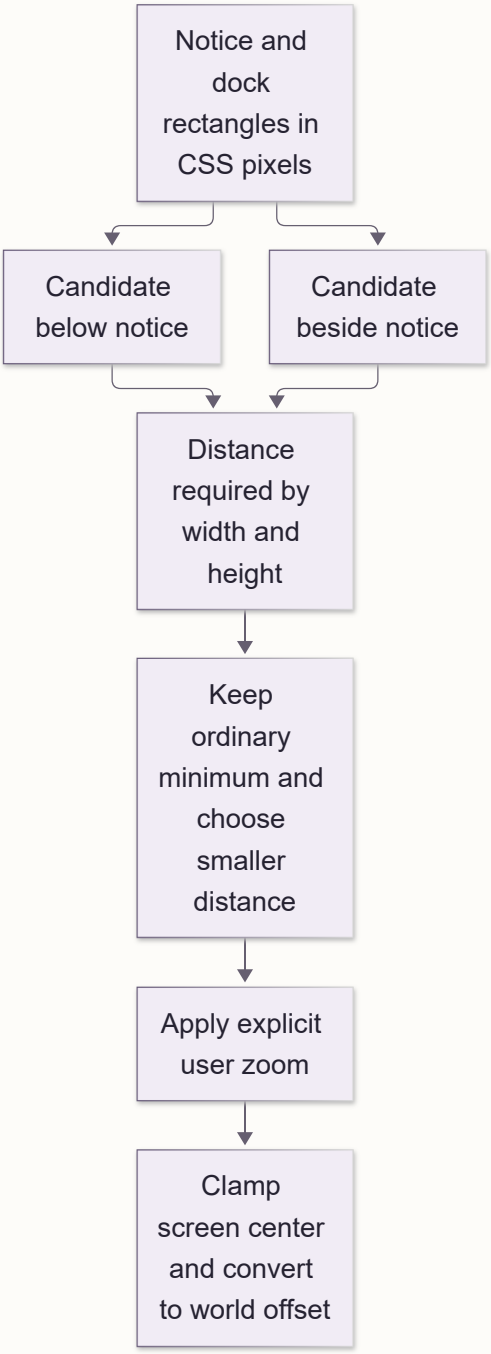


Figure 07-01. A two-candidate optimization, not a general packing solver. Default fitting and deliberate zoom have different contracts.

The scene compares the below and beside candidates, includes its ordinary minimum distance and selects the smaller fitted distance. Fewer world units away means a larger useful console. It reserves dock/hint space and observes notice-host size changes rather than reading layout every frame.

The original below-only fix shrank the console too far in short landscape. The next version accidentally applied an unzoomed minimum after the gesture, canceling explicit pinch zoom. The correction fits the baseline first and then multiplies by `this.zoom`. Default framing stays clear;

deliberate inspection zoom can still crop. Thus "fully visible at default zoom" does not become "users can never zoom in."

Screen Y grows downward; world Y grows upward. Moving the camera/look target requires the correct sign to move the projected console into the safe region. Actual projected geometry caught the initial sign error.

7.4 Linear-light sampling and time-based convergence

A 24 by 16 sample has 384 pixels. A 256-entry lookup table converts channel values to linear light; three sums yield mean red, green and blue. Work is `O(384)` per sample and constant relative to the native framebuffer size. Live resampling is limited to 125 ms intervals.

The convergence factor is `1 - exp(-18 * deltaSeconds)`. Unlike a fixed "move ten percent per frame" rule, it uses elapsed time, so refresh rate does not directly set the adaptation speed. A settling threshold ends imperceptible drift. This is smoothing, not recovery of missing native frames.

7.5 Smooth transitions and a sliced wordmark

The cubic easing curve `f*f*(3 - 2*f)` has zero slope at both endpoints. It interpolates insertion position and view transitions without abrupt start/stop velocity. Quaternion interpolation handles orientation.

The startup wave divides the wordmark into narrow source columns, offsets each vertically by a sine and draws them to a higher-resolution canvas. It deforms the supplied lettering instead of replacing it with a font. Reduced-motion mode uses a settled unsliced presentation.

Exercise. A safe region is half as wide but equally tall. Which fit term changes, and why is fitting only the console center insufficient?

Answer. The width-required distance doubles before other constraints are considered. A center inside the region says nothing about the envelope edges; fit size first, then clamp the center with half-envelope margins.

7.6 Alpha contours and nested engraved surfaces

The later rear-logo delta reuses the A/B marching-contour mechanism. Scan adjacent alpha samples, interpolate threshold crossings and connect grid edges through a map. Every recovered vertex must have two neighbors; an invalid loop reports an explicit error. Keep A/B's threshold128and original coordinate projection; the rear mask instead uses16to include colored orbital strokes, with separate physical sizing.

Sort rear contours by absolute area. Bounding boxes reject impossible parents; a point-in-polygon crossing test identifies the smallest enclosing prior loop. Even nesting depths belong to colored cavities; odd depths are plastic islands. Attach each child as a hole in its parent, then triangulate

both alternating surface sets. Normalize winding so the shallow wall normals face the cavities.

Worked example. An opaque ring has an outer contour at depth0and a transparent counter at depth1. Cut the outer loop out of the cover, but restore the counter as plastic. An additional colored island at depth2cuts only that counter. Painting a rectangle or treating every loop as a hole would lose this topology.

Raster scanning and loop traversal cost O(W*H+V), with O(V) contour storage. Ordering C contours uses area-based comparisons; naive containment can cost O(C*V) in the worst case before triangulation. This happens once during model construction, not per frame. The mask only defines geometry: UVs sample the unchanged full color texture, keeping color preservation separate from cavity construction.

Source trail: S-02, S-03, S-05, S-06, S-07, S-08 and S-09.

Second-edition practice: reason with units and bounds

CH07-W01 - A narrower available region

Use the chapter's perspective approximation with envelope width 6 world units, viewport height 800 CSS pixels, `tan(fov/2) = 0.5`, and available width 600 CSS pixels. Required distance is `6 * 800 / (2 * 0.5 * 600) = 8` world units. The units cancel correctly. The height constraint and ordinary minimum still need separate evaluation; this single term is not the final camera.

CH07-R01 - Halve only the width

At 300 available CSS pixels the width distance becomes 16 world units. The console must be farther away unless another layout candidate offers more usable width. Do not halve its mesh dimensions: that would also change the physical relationships with the rest of the scene.

CH07-Q01 - Does a heap improve four slots?

Explained answer

The newest-save selector sorts four candidates. Should it become a persistent priority queue to improve asymptotic complexity?

CH07-A01 - Optimize the real workload

Back to question

Not for this bounded use case. Sorting four items is simple and cheap, while a heap adds mutation and ordering invariants. `O(k log k)` describes growth; it does not by itself establish a performance problem at `k <= 4`. Reconsider only if the product changes the actual workload.

PART 3 / ENGINEERING THE EVIDENCE

CH08

Design principles and trade-offs

> *Choose boundaries by their recovery benefit and real cost.*

Before reading
Chapters 01-07.

Contents

Design principles and trade-offs

> Adopted first-edition teaching text; dated later notes remain historical. New practice follows. Current status: orientation and Chapter 13.

Principles are useful here only when tied to an actual failure or ownership decision. This is not a catalogue of patterns applied retroactively by name.

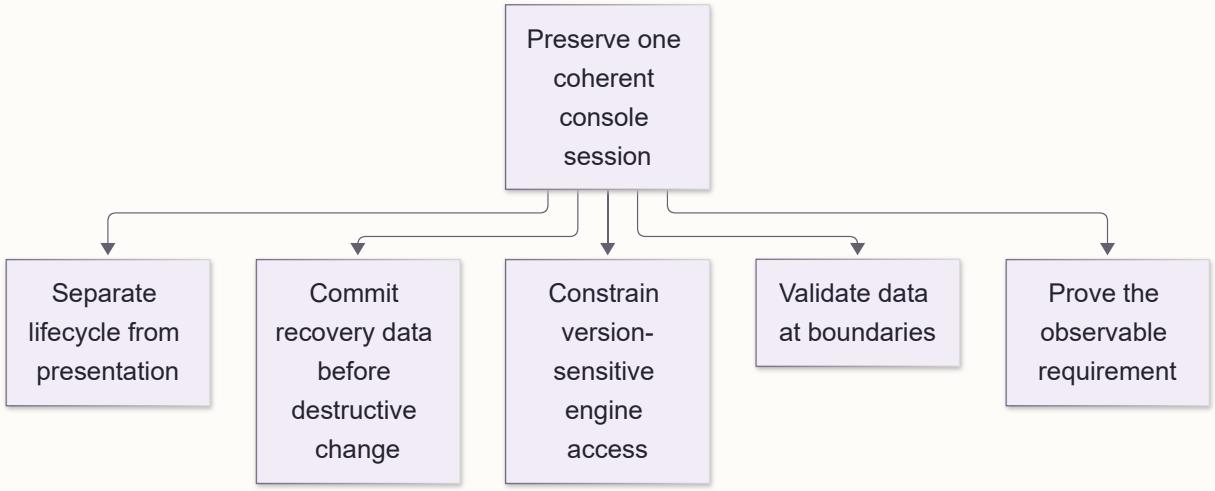


Figure 08-01. Each principle serves session coherence. None independently establishes security, permanence or deployment.

Single ownership reduces conflicting behavior

The host owns progress, keyboard mapping and cartridge lifecycle. The adapter owns native integration. The scene owns geometry and lighting. Managed native settings prevent an upstream menu from independently re-enabling a second save system or virtual gamepad.

The trade-off is more explicit integration code. There is no generic message bus framework or server layer merely to express these boundaries. Small modules and checked commands are enough for one active cartridge.

An adapter contains change; it does not eliminate dependency risk

Pinned native internals include state capture, filesystem access, setting changes, VSync and retained framebuffer behavior. Concentrating them in `player.js` and the audio adapter makes upgrades reviewable. It does not make an internal upstream API stable or certify the CDN's bytes.

The iframe allows complete teardown when a cartridge changes. Because it is same-origin, it is not protection against the runtime reading that origin's storage. Confusing lifecycle containment with hostile-code isolation would misstate the design.

Fail before discarding the escape route

ROM validation precedes replacement. Storage commit precedes power teardown. Backup validation precedes restore. A failed operation retains an explicit error rather than silently using an empty record and announcing success.

This ordering is a practical recovery principle, not a claim that the entire UI is one ACID transaction. Native execution, IndexedDB and DOM state span different systems. The implementation uses bounded operations and honest failure states instead of promising global atomicity it does not possess.

Presentation should not mutate simulation

Lighting, pickup, inspection and fullscreen change how the machine is seen. They do not require a second core, a ROM reload or a save-state round trip. The shared native picture and shared bezel derivative avoid divergent versions of the same screen.

The intentional exception is power: it is a lifecycle operation, not a visual toggle. Preserving that exception is more important than forcing every button through one simplistic "show/hide" abstraction.

Correctness follows the named quantity

A lit cap is not proof of native input. An enabled speed flag is not measured acceleration. A bounding box is not visible shell geometry. A matching public hash is not human approval. Tests become stronger when they measure the requested boundary rather than a convenient proxy.

The notice revision is a useful counterexample to over-constraining a system. Its first safe-fit minimum protected default clearance but destroyed explicit pinch behavior. The final principle is narrower: preserve default fit and honor deliberate user zoom. More constraints are not automatically more correct.

Exercise. Should a save-store read failure return an empty game record so the player can proceed without seeing an error?

Answer. Not as an unannounced substitute. It would make existing progress look absent and could mislead later writes. Report storage failure and explain session-only/export recovery. Any continued play must not claim persistence.

Source trail: S-01 through S-04, S-06 and S-11.

Second-edition practice: expose the failed alternative

CH08-W01 - Restore without recapturing stale state

A validated checkpoint represents the exact intended instant. Persisting a fresh capture immediately after an asynchronous restore may instead observe an older or unsettled frame. Preserve the exact validated checkpoint as the autoslot when that is the required recovery behavior. This is an ownership decision about which bytes are authoritative, not a preference for fewer calls.

CH08-R01 - Make the native interface stable

Suppose an upstream public API eventually provides the same capability. The adapter still owns translation and error handling, but fewer internal assumptions may be necessary. Replace an integration mechanism because the contract changed, not because a design-pattern name requires an adapter forever.

CH08-Q01 - Why not one giant transaction?

Explained answer

Can the host promise atomicity across native execution, DOM state and IndexedDB merely by placing their promises in one async function?

CH08-A01 - Ordering is not distributed atomicity

Back to question

No. Those systems have different commit and rollback behavior. The application can validate first, persist before destructive action and retain recovery routes. An async function sequences work but does not create shared rollback. Document the actual failure states rather than naming the whole operation ACID.

PART 3 / ENGINEERING THE EVIDENCE

CH09

Testing and delivery

> *Separate implementation, qualification, deployment and acceptance.*

Before reading
Chapters 01-08.

Contents

Testing and delivery

> Adopted first-edition teaching text; dated later notes remain historical. New practice follows. Current status: orientation and Chapter 13.

Build a ladder of evidence

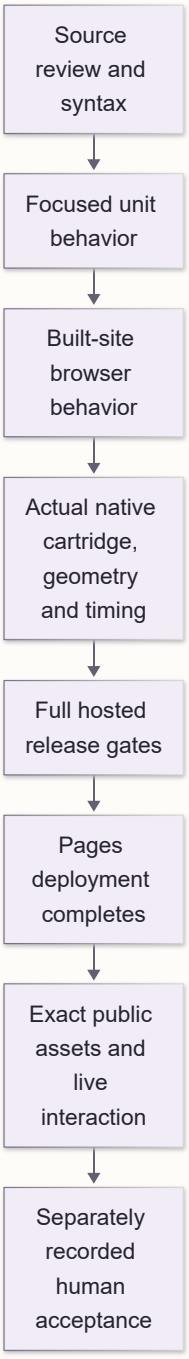


Figure 09-01. Later layers answer different questions. A lower-layer PASS cannot be relabeled as deployment or user approval.

The unit suite covers pure validation, backup, preferences and related application logic. Browser tests load the built site under the actual project prefix. Native cases run the original Pixel Lab cartridge through real EmulatorJS/mGBA; a shell screenshot alone would not prove native gameplay.

The full workflow has 11 unit cases and 39 browser cases across three 13-case shards at this source revision. These counts describe the actual edition, not a minimum that future tests must artificially preserve.

Prefer a measurement over a flag

The speed regression uses the original cartridge's clock and wall time. The VSync fix reconciles the native loop rather than merely changing an icon. A hosted 2x sample later reported 4.5609x, above its unchanged 2.6x bound. That observation is recorded without inventing a root cause or raising the bound until the gate passes.

The notice regression measures actual visible handheld geometry, a separate 44px power target, dock clearance, computed glass styling and ordinary-fit restoration. It includes five viewport shapes, a real invalid-file error and an explicitly synthetic long-text fixture. Synthetic text proves wrapping and bounded scrolling, not that a real native error produced that wording.

The later workstream 33 audio investigation shows a similar coverage gap. Surviving one startup stall did not prove sustained buffering: actual private playback exposed repeated roughly 100 ms underrun/restart gaps as the startup cushion drained. The correction increases native buffer capacity, not speaker volume or game speed. Its regression checks real allocated PCM budget as well as the requested setting, and extended playback measures scheduling gaps. Keep that later evidence separate from this book's pinned source edition. Neither PCM measurements nor a green test establish human hearing quality on every audio device; a larger reserve also trades additional audio latency.

The first public follow-up also catches a measurement mistake: allocated native slots receive PCM lazily, so summing populated buffers can understate capacity even after a bounded wait. Check the actual slot count and uniform positive block size instead. The unchanged capacity bound still rejects the old source. Distinguish reserved capacity, currently populated data and scheduled playback headroom; those are three different quantities.

Workstream 44 later reproduces starvation even with the larger allocation. An audio-only recording analysis and actual private playback lead to a clock comparison, not another capacity increase. Removing repeated recovery pre-roll worsens starvation; native VSync-off instead maintains 161-206 ms of live queued sound in a 90s control. Default to existing native audio synchronization while retaining explicit VSync choices. A new original-ROM regression measures real scheduled gaps, steady-state hitch tolerance and the SRAM cartridge clock. Four audio cases no longer need framebuffer screenshots for readiness. This extends the earlier evidence rather than retroactively erasing its limits. The added case makes the later suite 40 cases. Its first hosted run encounters multi-second software-GPU draws rather than the intended bounded 40ms hitch. Isolate graphics cost as the existing native clock regression does, retaining all audio bounds; the old clock still fails measured reserve. Full-view private playback remains separate evidence rather than claiming this isolated fixture qualifies an arbitrarily slow renderer. Fresh normal `42008857811` then passes all40cases and deployment. All28public assets match; four actual public audio cases pass45.2s with >=189.9ms steady reserve, a40ms hitch leaving154.2ms, zero gaps and a60.05fps SRAM clock. Keep that public qualification distinct from the private full-view180s run.

Workstream 34 deliberately reverses an earlier product requirement: the user now permits notices to overlap the handheld. Replace notice-separation checks with ordinary camera-position/look-target invariants, while retaining viewport fit, toolbar clearance, glass styling and dismissal checks. An intentional behavior change should update its specific contract, not weaken unrelated gates. The first hosted run then exposes a directly dependent user flow: the permitted power-on notice intercepts a real shoulder-button click. Dismiss it through its real close button before exercising every native input; retain the assertions and avoid forced clicks. The fresh normal workflow and both actual public overlay/input cases pass. A changed visual contract can require an explicit interaction step without justifying weaker functional coverage.

Let failure teach the requirement

The initial camera shift had the wrong vertical sign. Clearance tests caught it. A below-only solution then passed geometry checks but made short-landscape play unnecessarily tiny; actual visual review caught the usability trade-off.

Explicit SwiftShader later exposed a stale camera read immediately after a notice moved into the drawer. The corrected test polls the unchanged ordinary camera formula instead of blessing the sampled stale value. Timeouts and clearance requirements were not loosened.

The first hosted notification run passed 37 of 39 cases and blocked deployment. It found a genuine pinch regression as well as the separate speed outlier. Applying zoom after baseline fitting repaired the actual interaction. A fresh normal run then passed all 39 cases. This causal sequence belongs in the ledger; replacing it with "all tests pass" would erase useful engineering knowledge.

Deployment is another boundary

At book authoring, corrected run `41875740589` has successful build and browser jobs but a queued deployment. There are no deployment steps establishing a published result. An empty pending-approval list does not identify a queue cause. The public asset receipt still identifies the older application.

That observation is historical. Before finalizing the edition, the same healthy run completes successfully; all 28 public assets match and five real public native/notice/pinch/fullscreen/save/input cases pass in 2.0m. Desktop/phone public notice captures are reviewed. Waiting and verifying closes the release boundary; it does not change the earlier local recordings' capture origin.

The repository's recovery policy is to start a fresh normal workflow after a failed run, never retry the failed attempt. A healthy queued deployment is not itself a failure and does not justify duplicate dispatches, dummy commits or weaker gates.

After deployment, independently compare the complete public payload with the intended build and exercise the actual public application. Do not infer release identity from a successful job title alone. Documentation-only changes can be checked as published repository blobs without

claiming the app redeployed.

Exercise. The corrected workflow's tests pass while deployment is queued. Can the README describe local footage as a capture from the new public release?

Answer. No. Label it an actual-build local preview and retain its capture origin. Later byte matching can qualify the deployed source, but it does not retroactively change where the recording was made.

Source trail: S-11 and S-12; current verification evidence.

Second-edition practice: identify what passed

CH09-W01 - A green earlier source

Commit A passes all hosted gates. Commit B changes rendering and its current reload-seating case fails. A remains evidence for A. B does not inherit that qualification because most files are unchanged. Keep implementation identity, test environment and release identity together in the receipt.

CH09-R01 - More shards, same failing condition

Split 56 cases across nineteen isolated runners. If one unchanged acceptance condition fails, deployment must still stop. Isolation can reduce contention; it does not prove a timing cause or justify ignoring the remaining failure. Chapter 13 describes the current exact-source release guard.

CH09-Q01 - What does matching a hash prove?

Explained answer

Every public application asset matches the intended build. Does that establish native audio quality, phone ergonomics and human aesthetic acceptance?

CH09-A01 - Identity is one layer

Back to question

No. It establishes byte identity for the compared payload. Live behavior, environment-specific qualification and human judgment answer different questions. Combine those records when the acceptance criteria require them; do not make a checksum carry claims it cannot support.

PART 3 / ENGINEERING THE EVIDENCE

CH10

Visual documentation and the development loop

> *Keep living guidance, frozen editions and causal history coherent.*

Before reading
Chapter 09.

Contents

Visual documentation and the development loop

> Adopted first-edition teaching text; dated later notes remain historical. New practice follows. Current status: orientation and Chapter 13.

Documents serve different readers

The README is a visual invitation. The guide teaches actions. Architecture explains ownership and contracts. This book teaches the mechanisms in sequence. The ledger explains why changes happened. TODO files define bounded work and completion criteria. Private continuity helps the next session resume; it is not public product documentation.

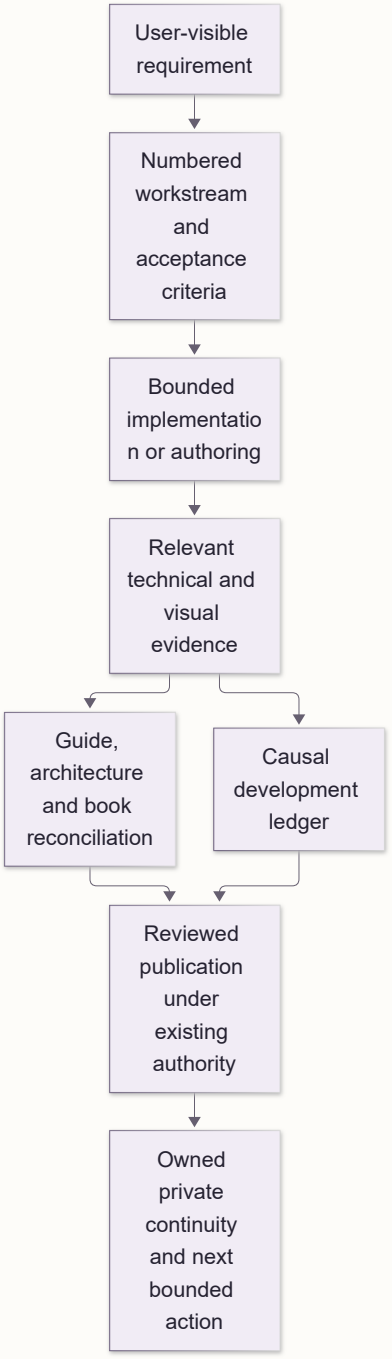


Figure 10-01. A complete development loop records both the result and the reason. Publication remains an explicit boundary.

Capture an action, not an invented demonstration

`capture-readme.mjs` drives actual browser controls, records a WebM and writes timed clip boundaries. It uses the original Pixel Lab demo. Pickup, inspection, startup, cartridge insertion and light changes are actual application states, not frames assembled to imply an interaction that never occurred.

The encoder uses those boundaries to create bounded GIFs at 10 fps and 800-pixel width. That is a repository-media trade-off, not the app's native render rate. The transparent orbital banner has a separate generator; refreshing interaction clips need not alter its approved alpha, composition or spacing.

The capture origin matters. A `BBA_CAPTURE_URL` override can record a local build while production is queued. Such footage can document implemented behavior, provided the caption and receipt do not call it a public capture. Its source identity, actual URL, timing receipt and page errors belong in Media. The book never turns a local receipt into production evidence.

The later BowserBOY refresh records those same interactions on the actual public site. The capture checkout is a documentation revision; independently matching the public application's assets to qualified `f5bc639` is what binds the footage to deployed source. Current `readme-*` stills can replace the visual front page without overwriting pinned first-edition geometry figures. The chrome website logo and purple console logo are separate supplied assets, not two color variants of one interchangeable global brand.



Figure 10-02. Actual startup motion from the documentation-refresh recording. See the current capture origin and edition in Media; this is not a BIOS movie or a rendered mockup. Print still: frame zero of the animated source; not motion evidence.

Review efficiently, but review the right thing

A GIF's first frame may identify its subject, yet a claim about a transition requires seeing the relevant motion. A labeled first/middle/end contact sheet helps find clipping and composition changes without repeatedly decoding every frame. Additional samples should answer a concrete uncertainty, such as whether the back or lights-off state is actually visible.

Hashes establish byte identity. They do not establish that orbit stars are uncropped, button relief looks manufactured or a notification is readable. Likewise, a correct diagram parser does not establish a readable chapter.

Workstreams and causal history

Each numbered `.todo` keeps Status, Milestone, Dependencies, Purpose, Tasks, Acceptance criteria, Completion evidence and Completion log. The index is navigation, not a substitute for those fields. Actual prerequisites are listed; file order does not create a dependency.

A useful ledger entry preserves the request, attempted approach, observed failure, correction, reason, evidence and remaining boundary. In this project, the notice sign error, landscape shrinkage, stale observer read and pinch regression are distinct lessons. They should not collapse into one vague "layout fixed" entry.

The first book's creation also corrects a documentation gap: earlier guides and journals existed, but numbered teaching chapters did not. The earlier incomplete-book note remains valid history. This edition explicitly supersedes that present-state claim; it does not pretend a book existed all along.

Exercise. A workstream's code is committed and all browser cases pass, but its acceptance criteria include independent public verification. Is it DONE?

Answer. Not yet. Record completed implementation and qualification separately, retain the release criterion as pending and preserve the exact next action. Publication queue time is not a reason to fabricate completion.

Source trail: S-13; Media, TODO index and development ledger.

Second-edition practice: keep several kinds of truth

CH10-W01 - A historical rejection remains useful

A first camera fit clears a notice but makes landscape play too small. The final implementation changes the baseline rule. The living guide should teach the final interaction; the ledger should preserve the rejected attempt and why it failed. A frozen book should retain its source identity and mark later changes rather than silently pretending it always described the new rule.

CH10-R01 - Change only the lamp

A lamp replacement invalidates captures containing the old lamp, not every unchanged logo asset or frozen milestone package. Inventory affected media, recapture actual interactions and bind the new bytes to their origin. Cache-busting a URL without replacing old pixels cannot satisfy the request.

CH10-Q01 - Can continuity be pasted into a public book?

Explained answer

Private session notes provide convenient chronological detail. Should a build recursively import them to make the development story more complete?

CH10-A01 - Audience is a boundary

Back to question

No. Public documentation needs deliberate, independently suitable source selection and authored explanation. Private continuity contains a different audience and potentially unrelated material. A reviewed source allowlist is both more reproducible and safer than importing everything a process can read.

PART 4

Current lessons and transfer

Preserve sharpness, material intent and exact-source delivery.

1. Sharpness without confusing the clocks
2. Warm fabric and a genuinely green indicator
3. Parallel tests and an exact-source release
4. Change one appliance without losing its promises

Picture the boundary. Explain the mechanism. Test the promise.

PART 4 / CURRENT LESSONS AND TRANSFER

CH11

Sharpness without confusing the clocks

> *Explain paint scheduling without claiming a timing diagnosis.*

Before reading
Chapters 05 and 07.

Contents

Sharpness without confusing the clocks

> Second-edition explanation of current source. No new runtime qualification is implied.

The question: can a slower desk still run a correct game?

The current scene preserves `min(devicePixelRatio, 2)`. It responds to expensive GPU work by changing paint cadence, not secretly reducing resolution. This is an important distinction: visual detail, simulation rate, input response and wall-clock deadlines are different quantities.

This chapter extends Chapters 05 and 07 with the actual current scheduling mechanism in `Scene.render`. Its explanation is source-backed; it is not a new benchmark or a claim that the latest hosted timing failure has been solved.

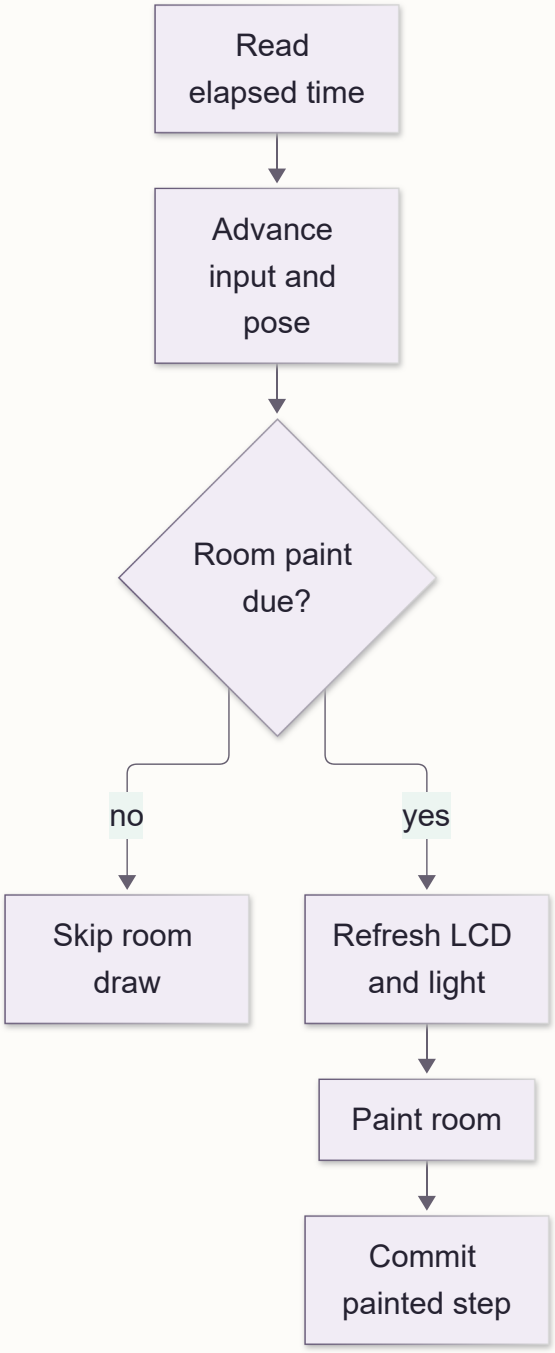


Figure 11-01. Skipping a room draw is not a command to pause the native machine. Visible insertion progress has a paint-commit rule of its own.

Three clocks and one visible claim

Wall time measures deadlines. The native runtime advances emulation according to its own scheduling and synchronization. Scene callbacks update physical feedback and decide whether to paint. Calling all three "frame rate" makes a diagnosis ambiguous.

The scene can skip the tabletop while screen fullscreen is active. It also avoids room paints while native work is marked busy or a modal dialog is open. After a slow rendered interval, its recovery decision can defer another draw. The current code selects a larger recovery bound for settled play than for transitions or cartridge motion. These are presentation policies, not a license to stop accepting releases or falsify native speed.

Button compression uses elapsed-time damping before the paint-due return. The physical pose can therefore keep approaching its bounded pressed state even when the expensive room draw is skipped. This is why a visual-feedback test must distinguish state evolution from the instant pixels were presented. An independent native-input test is still required.

Why insertion cannot simply jump to elapsed wall time

The cartridge animation is intentionally visible. If a browser stalls for several seconds and its next callback immediately marks insertion complete, the user may never see meaningful insertion motion. The current scheduler computes an eligible step only when a paint is due, caps that step at 500 ms and commits motion timing after an actual draw. The early camera motion shares that clock.

This protects a visible sequence, but creates an engineering trade-off: several actual paints are necessary to complete it. A wall-clock test that waits fifteen seconds still expects seating within those fifteen seconds. The code's mechanism explains why a paint schedule is relevant; it does not by itself establish why a specific hosted run missed that deadline.

The latest recorded hosted failure is exactly that unresolved boundary. Do not announce a root cause from a plausible reading of a condition. Measure draw opportunities, committed insertion steps and dataset transitions on the failing path before changing the implementation.

CH11-W01 - Compute a visible step budget

Use a synthetic clock with insertion delay 850 ms and motion duration 1350 ms. The total accumulated animation time needed is 2200 ms. If every eligible draw contributes the maximum 500 ms, at least five such steps are needed from zero. That is a lower bound under the stated assumptions, not a measured number of callbacks on a real host.

Four maximum steps produce 2000 ms. The motion fraction is then `(2000 - 850) / 1350`, about 0.852. Cubic easing maps that fraction to about 0.941 of the distance. The cartridge may look almost seated while its logical completion predicate still correctly says `inserting`.

CH11-R01 - Add a modal interval

Now place a dialog over the scene for part of the wall-clock observation. A paint-free interval cannot be counted as visible insertion progress under this rule. The changed condition alters the available committed steps, not the emulated cartridge's CPU speed. A useful diagnostic records dialog state, paint eligibility and accumulated motion time separately.

If the acceptance criterion requires insertion to finish while a modal is open, that is a product-contract decision with consequences for visible animation. Do not quietly reinterpret the existing criterion after a failure.

CH11-Q01 - Pick the honest optimization

Explained answer

A software GPU is slow. Which proposed shortcut preserves this project's current contract: halve pixel ratio, increase every deadline, or investigate bounded scheduling while retaining resolution and assertions?

CH11-A01 - Preserve the measured requirement

Back to question

Investigate scheduling. Halving pixel ratio changes sharpness; larger deadlines change what the suite accepts. Neither proves the named behavior works as required. A scheduling change is not automatically correct either: it must preserve native progress, responsiveness, visible motion and the unchanged timing bound. The failed source remains unqualified until those observations pass.

What you can now diagnose

When a game sounds steady but the console moves in visible jumps, you can ask separately about native reserve, room draws and presentation timing. When insertion looks almost complete but the dataset is still `inserting`, you can inspect the completion predicate instead of calling the test flaky from appearance alone.

Source: current `src/scene.js`, especially `render` and `insertCartridge`; historical performance contracts in `docs/ARCHITECTURE.md`; exact current failure in `docs/VERIFICATION.md`. The edition's claim register binds these paths to source and evidence epochs.

PART 4 / CURRENT LESSONS AND TRANSFER

CH12

Warm fabric and a genuinely green indicator

> *Choose the responsible layer for texture, seams and exposure.*

Before reading
Chapters 04, 05 and 11.

Contents

Warm fabric and a genuinely green indicator

> Second-edition explanation of current source. No new runtime qualification is implied.

The question: why can the correct material still look wrong?

A white shade can appear dirty if low-frequency brown variation is painted into its texture. A finely woven material can disappear if its luminous term is uniform and dominates reflected light. A green indicator can turn nearly white when dark adaptation raises exposure. These are different failures at different stages of the image pipeline.

The current lamp is an independently modeled STORSEGEL-style white drum shade and ash tripod, with hidden wiring. That is a visual approximation in the project, not an IKEA asset, product endorsement or calibrated digital twin.



Figure 12-01. Current README still from the final November 6 capture edition. The local capture origin is retained in Media; later public byte matching qualifies application `1b0385b`*, not every future source.*

Separate shape, material and emitted energy

The open cylinder gives the shade its actual silhouette. Procedural warp and weft create a fine neutral intensity pattern, not large brown stains. The same texture modulates both the material map and emissive map. That second connection matters: when the shade glows, the luminous term still carries the weave instead of washing it into a solid bright cylinder.

Warmth comes from the chosen emission and light colors. A cream emission is not a reason to tint the clean underlying fabric with irregular grime. Geometry and material construction use a meters-per-unit conversion so the lamp's proportions remain coherent with the handheld and table.

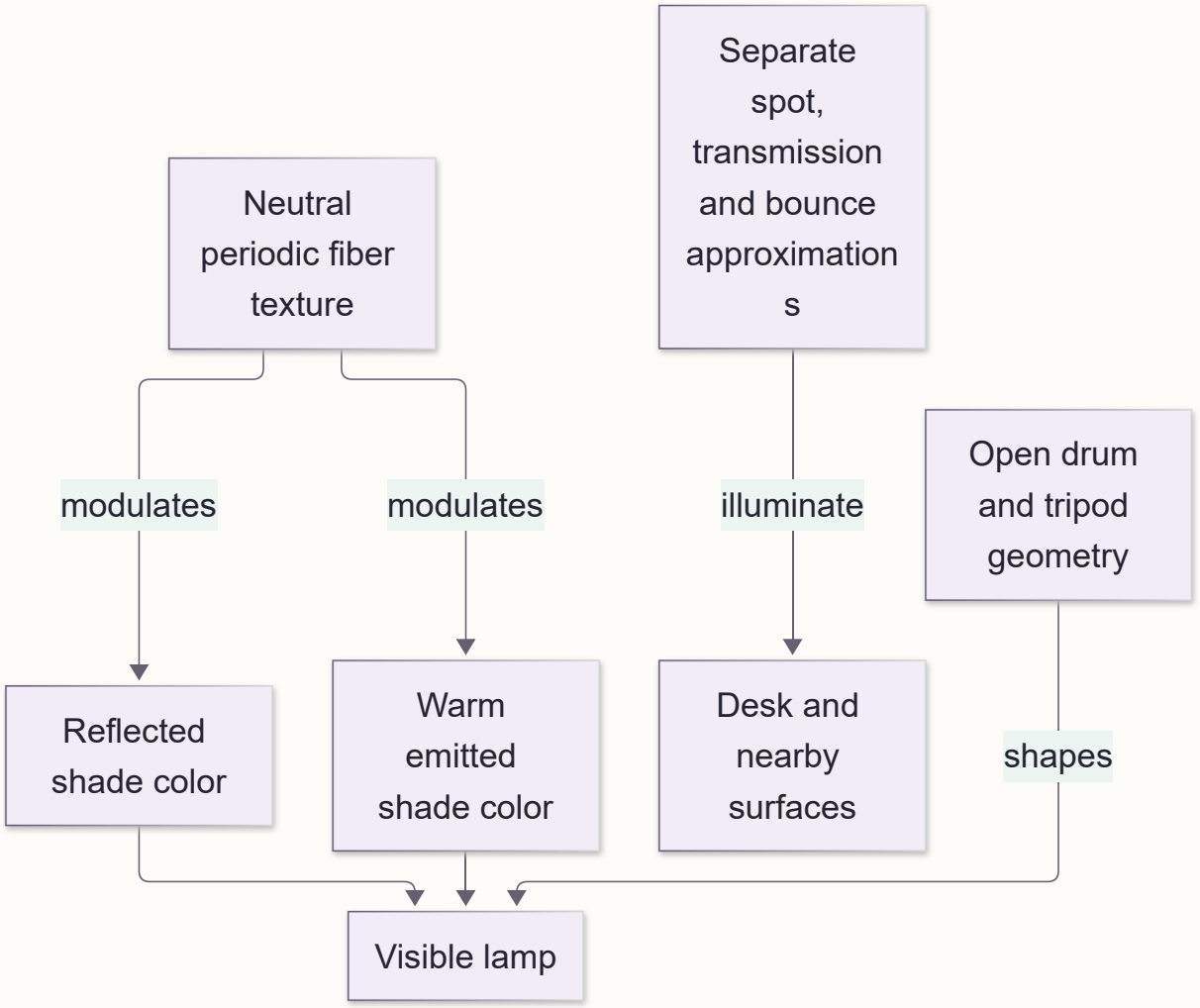


Figure 12-02. Texture detail belongs to both shade terms. Actual room illumination uses separate light approximations; an emissive material alone is not a complete global-illumination solution.

A seam can be a placement problem

An area emitter crossing the shade can create a hard lighting seam. Making the entire scene dimmer hides the symptom but changes unrelated surfaces. The implemented correction places the transmitted-light plane outside the fabric. A separate downward area source reveals the tripod through the open bottom, and a finite upward patch approximates the first warm table bounce.

These are deliberate real-time approximations. They do not model every fiber's transmission, spectral scattering or multiple bounce. Their usefulness is judged against the visible requirement and independent light controls. The lamp, room illumination and content-driven LCD remain separately switchable; an LCD-only capture must not secretly add fill lighting.

Exposure is a system-wide multiplier

The scene adapts between bright-room exposure 0.018 and dark-room exposure 0.45. That is a factor of twenty-five. Leaving a highly emissive indicator unchanged under this increase can push it into a bright, desaturated result. A source color labeled green is not proof of green pixels after tone mapping.

The current correction compensates only the lit indicator's emission: `80 * 0.018 / currentExposure`. It preserves the established room adaptation instead of changing every surface to rescue one small emitter. The indicator also has a smaller physical size. Geometry, emission and final pixel color are therefore separate checks.

CH12-W01 - Preserve an emitter through adaptation

At exposure 0.018, the compensated emission is 80. At exposure 0.45 it is 3.2. In both cases the simple product is 1.44. This calculation explains the intended compensation; it is not a complete tone-mapping derivation. Reflected light, color management, material response and nonlinear display mapping still require actual rendered inspection.

CH12-R01 - Put the texture on only one term

Keep the weave on the reflected map but remove it from the emissive map. As luminous contribution dominates, the shade increasingly looks smooth. Adding dirtier contrast to the reflected map treats the wrong term. Restore the detail on the emitted contribution and inspect at the actual warm exposure.

CH12-Q01 - What proves LCD-only reflections?

Explained answer

A picture shows the console in a dark room. Is that enough to prove the LCD casts moving content-colored light on nearby surfaces?

CH12-A01 - Observe source and response together

Back to question

No. Record actual lamp and room switches off, the real LCD active, changing native content and motion of the console. Inspect the surroundings through the transition. A still can show one state; it cannot establish temporal response or rule out capture-only fill. The existing final media receipt describes that actual interaction without claiming path tracing or exhaustive frame review.

What you can now change safely

Choose the responsible layer first. A silhouette complaint calls for geometry. Uniform glow calls for material-term inspection. A seam calls for placement. Bleached green calls for the full exposure path. This avoids global lighting edits that accidentally destroy the user's approved room.

Source: current `src/lighting.js:createDeskLamp`, `src/lighting.js:ScreenIllumination`, `src/scene.js:render` and the final capture receipt in `docs/MEDIA.md`. No new media capture is claimed here.

PART 4 / CURRENT LESSONS AND TRANSFER

CH13

Parallel tests and an exact-source release

> *Bind selected bytes to qualification, naming and publication.*

Before reading
Chapters 09 and 10.

Contents

Parallel tests and an exact-source release

> Second-edition explanation of current source. No new runtime qualification is implied.

The question: which green result authorizes these bytes?

A release pipeline is not merely a command that uploads a ZIP. It is an argument that the selected source passed the required gates, that the package contains the intended files and that publication does not move an existing identity. The argument fails if evidence for one commit is attached to another.

This chapter explains the implemented manual workflow at `9ff921b`. It does not announce a successful release. The actual first dispatch was blocked by its qualification check, and that block is useful evidence of the safety boundary.

Parallelism changes placement, not acceptance

The current browser inventory is 56 cases across nineteen single-worker jobs: eighteen groups of three and one group of two. Retries remain zero. Cases must appear exactly once. Native, visual and protocol requirements remain intact; extra runners are not a reason to reduce rendering resolution, remove difficult cases or increase deadlines.

With perfect independence, wall time is approximately the longest shard plus setup, queueing and deployment. The sum of test durations is not that wall time. Heavy native startup, software-GPU contention and fixed browser overhead can prevent linear speedup. More jobs also consume more aggregate resources. Measure the actual critical path rather than claiming nineteen times faster.

The latest source still fails reload seating in one shard. Isolation is an implemented technique, not proof that every scheduling problem is solved. The earlier fully qualified visual release remains valid for its own bytes.

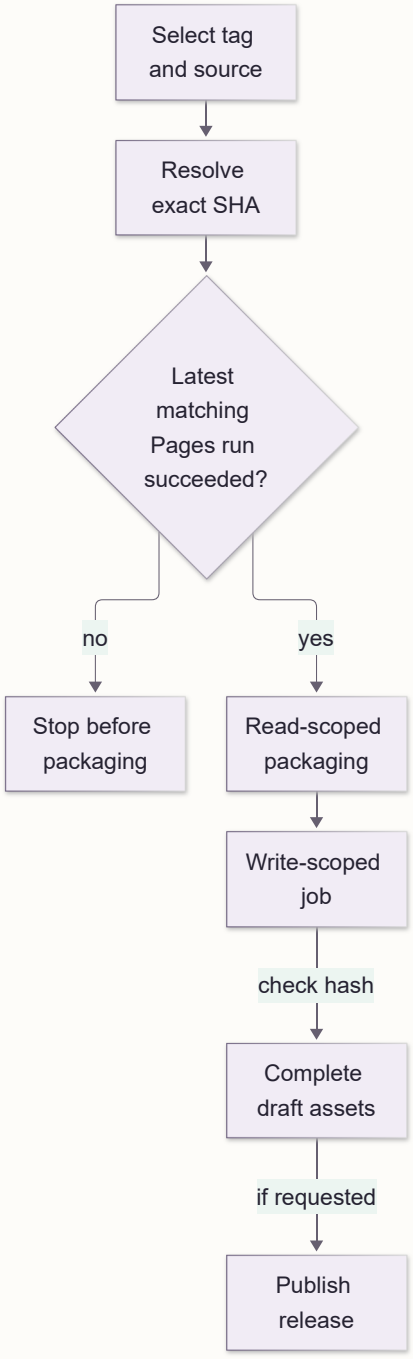


Figure 13-01. Qualification and publication are distinct jobs. A checksum protects artifact transfer identity; it cannot replace source qualification.

Resolve once, preserve existing names

The workflow accepts a version tag, optional source reference and draft and prerelease flags. It resolves the checkout's exact SHA. A tag must have the expected version form; an existing tag pointing elsewhere is rejected rather than moved. An existing release is not silently overwritten.

The qualification step asks for the latest existing Pages run for that exact SHA and requires completed success. A pending, failed or absent run cannot be replaced with "main was green yesterday." The workflow does not dispatch new verification as a side effect of this question.

The packaging job uses read permissions, runs the existing checks and builds the allowlisted static output. Its notes identify source and successful qualification. The archive contains the static site plus notices and notes, not the whole repository, book source, context store or ROM collection.

Grant write authority only where needed

The publication job downloads the completed artifact, verifies its checksum and has the scoped permission needed to create a GitHub Release. It initially creates a draft with all intended assets, then optionally publishes it. This ordering reduces the chance of publicly presenting an incomplete upload. It is not a universal atomic transaction: remote APIs and network failures still need explicit operational handling.

Concurrency is grouped by release tag without canceling an in-progress request. That controls overlapping attempts for the same identity; it does not create permission to move tags or bypass verification. A package that passes local validation is still not an externally published release.

CH13-W01 - An absent exact-source success

Suppose commit A has a successful Pages run, but the owner selects commit B. B's latest run is in progress. The correct release result is failure before packaging/publication. No source-specific success exists yet. The workflow does not guess that B will pass and does not quietly use A's package.

This matches the important boundary of the first real manual request: it stopped safely while its chosen source had no successful exact-source Pages qualification. A subsequent failed Pages result did not turn that earlier block into a defect in the guard.

CH13-R01 - A changed tag destination

Now B qualifies, but the requested tag already points to A. Should the pipeline move the tag to B because B is newer? No. Tag identity is part of the contract. Choose an appropriate new version through the owner's normal process. Qualification and naming are independent gates.

CH13-Q01 - Does a static ZIP include the native core?

Explained answer

The archive contains the allowlisted site. Does extracting it make native startup fully offline and preserve origin-local saves from the public site?

CH13-A01 - Packaging does not erase runtime boundaries

Back to question

No. The runtime still fetches the pinned upstream distribution. Saves belong to the browser profile and origin, so a different host or port has a different store. Serve the extracted package over HTTP, preserve notices and export saves before changing hosts. The package checksum identifies bytes, not network availability, licensing clearance or save migration.

What you can now decide

For a release request, you can name the exact source, exact qualification, exact tag and exact archive. If one is missing, say which one. A blocked release can be the correct implementation result while operational delivery remains incomplete.

Source: `.github/workflows/release.yml`, `.github/workflows/pages.yml`, `releases/README.md` and the dated receipts in `docs/VERIFICATION.md`.

PART 4 / CURRENT LESSONS AND TRANSFER

CH14

Change one appliance without losing its promises

> *Design a safe presentation change and defend its evidence.*

Before reading
Chapters 01-13.

Contents

Change one appliance without losing its promises

> Hypothetical capstone; not an implemented feature.

A deliberately hypothetical assignment

Design a change that adds a second presentation theme to the same GBA console. The new theme changes desk materials and lighting, not native execution, cartridge identity, save format, power semantics or input mapping. It must retain full-resolution rendering and an honest release gate.

This is a design exercise, not an implemented feature or a promise to add another console. In particular, "another theme" does not establish support for Nintendo DS or any additional emulation core.

Locate the boundaries before writing code

Begin with a state table. Which values are persistent user preferences? Which belong to the scene? Which native settings must remain unchanged? Decide how an old preference record without a theme field is interpreted without changing unrelated defaults. Decide how an invalid theme value is reported or validated through the project's existing preference conventions.

Next draw the change path. A theme selection should update materials and light configuration without tearing down the native child. The retained framebuffer remains the LCD source. Existing input ownership survives or is explicitly released according to current view-transition rules; it cannot be stranded because a modal swallowed keyup.

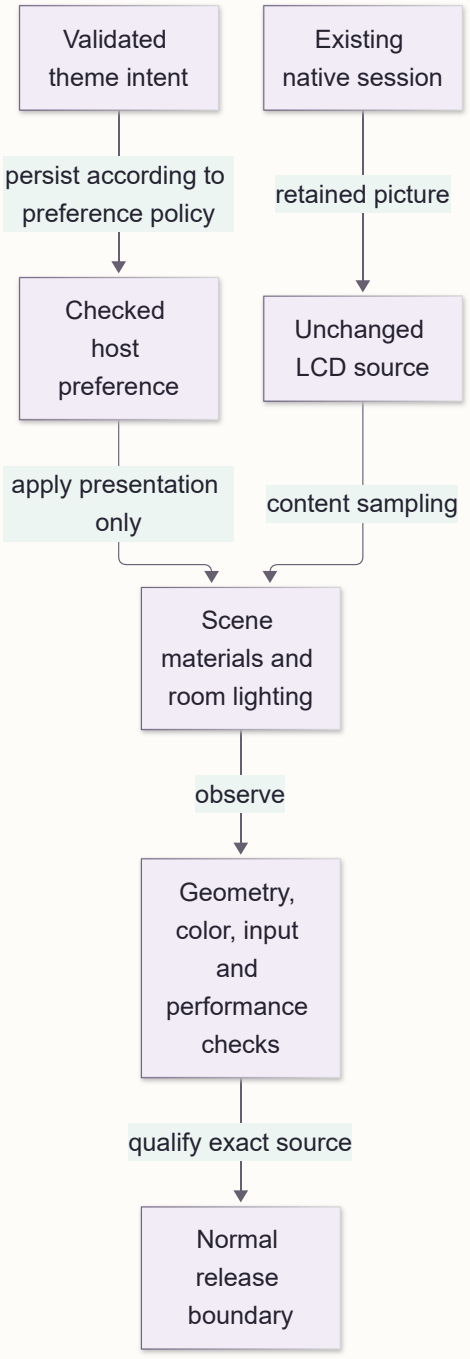


Figure 14-01. The proposed theme is intentionally outside native lifecycle ownership. This diagram is a design exercise, not an executed code path.

CH14-W01 - An acceptance table that can fail

Write four explicit claims. First, changing theme does not restart the game: observe native progress and session identity across the selection. Second, the same save remains associated with the same ROM digest: round-trip a checkpoint without changing its compatibility identity. Third,

lamp, room and LCD controls remain independent: inspect their actual state and rendered response. Fourth, the scene retains the pixel-ratio contract: measure the renderer, not only the CSS canvas size.

Add a failure case to each claim. A new iframe would violate the first. Using a theme-qualified ROM ID would violate the second. An unconditional ambient fill could violate LCD-only darkness. A hidden dynamic-resolution shortcut could violate the fourth. Acceptance becomes useful when the rejected implementation is concrete.

CH14-R01 - Change theme during an unsuccessful save

Now the user requests theme change while a required save is failing. Should the theme handler also own recovery and destroy the old child? No. Respect existing serialization and error handling, and leave the runtime recovery route intact. If theme changes are allowed during native work, document the presentation ordering; if not, use the established disabled/queued behavior. Do not invent a second lifecycle coordinator for a cosmetic feature.

The changed condition tests ownership under failure, not just normal visual appearance. A polished screenshot alone cannot establish that the ordering is safe.

CH14-Q01 - Produce a defensible delivery plan

Explained answer

Submit a short design with five parts: state ownership, one normal/failure diagram, source boundaries, focused regressions and release/evidence steps. Include a media-update decision and a rollback story. What would a sound answer have to defend?

CH14-A01 - An explained assessment rubric

Back to question

A strong answer keeps one native session, one content-addressed save identity and the established power/checkpoint distinction. It uses the existing checked preference and scene boundaries rather than a parallel settings system. Its failure path reports persistence errors without inventing success.

Its tests measure actual quantities: native progress/session retention, save identity, aggregate input release, independent lights, final pixels and unchanged render resolution. It does not replace these with object names or a theme flag. It states which environments were actually exercised.

Its delivery plan selects the exact implementation SHA, preserves all normal gates and independently verifies deployment if required. New media containing the theme is actually recaptured and labeled by origin. Old frozen editions and approved unchanged assets are preserved. Private notes do not enter the public allowlist.

Reasonable designs may apply the theme immediately or after an established transition, but must justify ordering and failure behavior. A design that reloads the ROM, silently discards a save error, lowers resolution or calls a local preview publicly deployed fails the core constraints regardless of its appearance.

Final synthesis

The important skill is tracing a user-visible promise to its owner, state transition, failure window and observable evidence. A browser console is a particularly compact laboratory because those boundaries meet in one object: a button, a pixel, a save, a light and a release all look simple until their clocks or meanings diverge.

You can now ask not just whether a feature exists, but whether these exact bytes deliver that exact behavior under the stated conditions. That question is the bridge between an attractive demo and a maintainable appliance.

Glossary and concept index

Term	Working meaning in this book	Read with
Adapter	Checked boundary around version-sensitive native integration; it contains assumptions, not all dependency risk	Chapters 02, 08
Aggregate input	An action is pressed while at least one source owns a hold	Chapter 06
Alpha contour	A boundary recovered from an image's opacity; nested loops distinguish cavities and islands	Chapters 04, 07
Battery data	The game's native save medium, used by a fresh boot	Chapters 02, 03
Checkpoint	An instant native state, separately restored rather than automatically substituted for physical power-on	Chapters 02, 03
Cold boot	Fresh native execution with available battery data, not a loaded checkpoint	Chapter 02
Commit boundary	The point at which the relevant system reports completion, such as an IndexedDB transaction completing	Chapter 03
Content address	A key derived from complete bytes; renaming does not change it, patching bytes does	Chapter 03
CRT preset	An upstream native shader choice, not a CSS approximation of its output	Chapter 05
CSS pixel	A layout unit; not necessarily one renderer or physical screen pixel	Chapters 04, 07, 11
Drawing buffer	The native canvas's retained rendered output, read later for presentation	Chapter 05
Emission	A material's luminous contribution; it is not by itself a complete room-light transport simulation	Chapter 12
Evidence epoch	The source/environment/cutoff to which a recorded observation belongs	Chapters 09, 13

Exact-source guard	A release check requiring qualification for the selected SHA rather than a nearby revision	Chapter 13
Exposure	A scene-wide brightness control whose interaction with tone mapping affects final color	Chapter 12
Folio	Printed page number in the rendered edition; it is not a stable chapter identifier	Maintenance reference
Framebuffer	The rendered native picture consumed by the LCD and fullscreen presentations	Chapter 05
Header validation	Bounded format checks; not proof that arbitrary ROM content is correct or harmless	Chapter 03
Integrity checksum	Detection of changed bytes; not an authenticated signature or encryption	Chapters 03, 13
Lifecycle isolation	Ability to replace/destroy a native instance independently; not hostile-code isolation	Chapters 01, 02
Linear light	Channel values suitable for energy-like averaging before display encoding	Chapters 05, 07
Native reserve	Scheduled sound ahead of playback; different from allocated buffer capacity	Chapter 09
Origin	Browser storage/security identity determined by scheme, host and port	Chapters 01, 03
Paint cadence	How often the expensive room image is drawn; not inherently the native simulation rate	Chapter 11
Pixel ratio	Renderer resolution scale relative to CSS pixels; this project retains its stated cap	Chapter 11
Projection	Conversion from world geometry through the camera to screen coordinates	Chapters 04, 07
Public byte match	Comparison identifying served payload bytes; it does not establish user approval	Chapters 09, 13

Recovery route	A remaining way to export or resume after failure; preserve it before destructive change	Chapters 02, 08
Runtime identifier	A save compatibility fence; not a universal promise for future native versions	Chapter 03
Session identity	Binding that prevents stale messages from one child/cartridge satisfying another	Chapter 02
Source allowlist	Deliberate list of permitted inputs; not a recursive import of the workspace	Chapters 10, 13
Static host	Server delivering files while the browser executes the application	Chapter 01
Tone mapping	Conversion of scene brightness/color toward display output; final pixels need inspection	Chapter 12
Workstream	Bounded task with purpose, acceptance, evidence and causal completion history	Chapter 10

Common distinctions worth keeping

Pause versus power. Pause retains the machine; power off is destructive after required recovery work. **Checkpoint versus battery.** Instant state and the game's own save data support different meanings of restart. **Allocated versus populated versus scheduled audio.** Capacity, filled buffers and live playback headroom answer different questions.

Implemented versus qualified. A code path can exist while its current hosted gate fails. **Qualified versus deployed.** A passed suite does not prove the site has those bytes. **Deployed versus accepted.** Technical publication does not manufacture human aesthetic approval.

Living versus frozen. A guide follows current behavior; a frozen book or release follows its own manifest. A newer result can supersede a present-state claim without making an earlier observation false.

A maintainer's field guide

Start with the failed promise

Use this reference after the teaching chapters. It is not a replacement for the player's task guide or for reading the relevant implementation.

Symptom	First boundary to inspect	Tempting but misleading shortcut
A renamed file seems to lose progress	Compare complete ROM digest, browser origin/profile and storage result	Match by filename or title
A power cycle loads an unexpected instant	Distinguish battery boot from checkpoint restore	Always load the newest checkpoint
A save appears successful but vanishes	Observe transaction completion/abort and actual error reporting	Treat `put` success as durable commit
A button stays down after a view change	Inspect held source sets, held-key mapping and release-on-blur paths	Clear a visual cap flag only
A stale response completes a new command	Check window, origin, session and request identity together	Compare only message type
Fullscreen appears different	Follow the shared retained native canvas and aspect ratio	Start another core for the new view
Scene is sharp but movement stalls	Separate native time, callback state and room-paint cadence	Reduce pixel ratio without consent
Cartridge almost seats but gate fails	Record committed motion steps and completion predicate	Decide completion by visual impression
Shade looks like a luminous cylinder	Inspect emissive map as well as reflected map	Add brown noise to the fabric
A dark-room indicator looks white	Inspect exposure and tone-mapped pixels	Assert that the source hex string is green
Audio has reserve but stutters	Separate allocation, live scheduled reserve and actual clock behavior	Add volume or repeatedly increase capacity
A release request is blocked	Resolve exact source, latest exact-source Pages result and tag identity	Borrow an earlier green commit's result
README still shows old geometry	Check actual media bytes, capture origin and served GitHub images	Change only the URL's version query

A small, honest investigation loop

1. Name the user-visible requirement and the exact source/environment.
2. Read its owner and directly affected state transitions.
3. Reproduce with the smallest meaningful fixture and retained bounds.
4. Record the observed failure before forming a cause.
5. Change the responsible mechanism, then rerun the affected check.
6. Reconcile documentation and delivery criteria without erasing history.

For timing failures, distinguish an observation from a hypothesis. "Expected seated, observed inserting after fifteen seconds" is an observation. "GPU contention caused it" requires additional evidence. A plausible mechanism from source is a reason to measure, not a completed diagnosis.

Documentation has several update rules

The README remains the visual invitation; its owner requested a minimal front page, so this edition is linked from the book/documentation hubs rather than expanded into a long front-page manual. The guide teaches actions. Architecture describes current boundaries. Verification identifies executed evidence. Media identifies actual captures. The numbered TODO and ledger preserve scope and reasons.

This rendered edition is frozen to its manifest. For a correction, keep chapter IDs, change canonical Markdown deliberately and rebuild the candidate. Check the source diff and all affected artifacts. A typography change can alter page numbers even when prose remains the same. A content change can invalidate companion page references. Never relabel an old companion as bound to a new PDF solely because the title matches.

The first Markdown edition has no page map. This edition's chapter-page table is therefore the first rendered map, not an invented old-to-new pagination concordance. Stable chapter IDs bridge the two presentations.

Local production and rights

The edition's source catalogue is explicit. Paths are confined to the repository and symlinks are rejected by the source resolver. Diagrams render locally; the browser denies external resource requests. That policy is a production boundary, not an operating-system sandbox.

The reader contains no scripts. Its internal links, rendered SVGs and image bytes must work in an offline standalone copy. The native PDF receives a concise outline and is checked for page geometry, searchable chapter text, learning markers and valid internal destinations. Browser layout and native page checks complement one another.

Selected native-page raster review covers composition and readability at identified samples; contact sheets do not prove every sentence was read. Neither mechanism establishes PDF/UA, printer prepress, independent media rights or human acceptance. Those remain separate questions for any proposed external release.

Editable cover artwork and production sources remain in the repository. Generated output, installed packages and private review images are not new canonical manuscript. Preserve notices for supplied artwork and dependency licenses, and do not package the whole workspace.

Source and claim register

Read source links as bounded evidence

The first ten adopted chapters retain their original source register. That register identifies the original `ab912d5` implementation and later dated deltas. It is not a claim that every sentence was re-executed on today's browser.

The new engineering chapters explain source at base revision `9ff921be5d1b1d4a6657a181fc3ac84ea72373e7`; relevant application and workflow files were unchanged by this book task. Documentation already contained uncommitted audit corrections. The source manifest hashes the exact selected files, so that distinction is not hidden behind a commit label.

Claim ID	Supported statement	Evidence and scope
C01	Static host delivers the shell; browser executes the selected native core	`src/app.js`, `src/player.js`, `scripts/site-files.mjs`; first-edition S01/S02/S10
C02	Save identity is content-based, with separate checkpoints and battery mode	`src/cartridge.js`, `src/save-store.js`; first-edition S03/S04
C03	Same-origin frame replacement isolates lifecycle, not hostile origin privileges	`src/app.js`, `src/player.js`; Chapters 01/02/08
C04	Held actions aggregate source ownership and release using the originally pressed action	`src/player.js`, `src/keyboard.js`; first-edition S02/S08
C05	Native post-shader picture supplies the LCD, fullscreen and reduced light sample	`src/player.js`, `src/scene.js`, `src/lighting.js`; first-edition S02/S06/S07
C06	Current scene retains capped device pixel ratio while scheduling room draws	`src/scene.js`, especially construction and `render`; Chapter 11
C07	Insertion has paint-eligible capped steps and a completion predicate	`src/scene.js:render`; explanation only, not a newly measured hosted diagnosis
C08	Woven detail modulates reflected and emitted shade terms	`src/lighting.js:createDeskLamp`; Chapter 12
C09	Indicator emission compensates current exposure during dark adaptation	`src/scene.js:render`; final-pixel correctness requires rendered evidence

C10	Manual release requires the latest successful existing Pages run for exact source	`.github/workflows/release.yml`; Chapter 13
C11	Release packaging and write-scoped publication are separate jobs	`.github/workflows/release.yml`; implemented procedure, not a completed publication
E01	Qualified visual `1b0385b` has recorded public byte/behavior evidence	`docs/VERIFICATION.md`, `docs/MEDIA.md`; historical release receipt
E02	Latest `9ff921b` Pages run fails reload seating and does not deploy	`docs/VERIFICATION.md`; dated audit cutoff, not a fresh status poll during book production
E03	First manual release attempt blocks before unqualified packaging	`releases/README.md`, `docs/VERIFICATION.md`; no GitHub Release claimed
E04	Final showcase media is from the November 6 local capture edition	`docs/MEDIA.md`; capture origin remains local despite later public application qualification
T01	Numeric worked examples and capstone are synthetic learning material	Explicit assumptions in the corresponding W/R/Q/A items; not benchmarks or implemented features

Exact public implementation references

- Scene at the current base
- Lighting at the current base
- Manual release workflow at the current base
- Qualified visual application
- Project dependency and artwork notices

Documentation links outside the compiled book open the repository's current public files. Those links are references, not embedded snapshots; the local manifest identifies reviewed input bytes. Upstream product guarantees require their own authoritative documentation, not inference from this application.

Selected images and excluded material

The manufactured-control image is a historical geometry figure. It is not a new lamp capture. The current tabletop still shows the final warm woven lamp. The startup GIF is represented by a labeled print still; animation itself is not proven by that still. Original supplied logos remain governed by the project notices. No new claim of independent artwork rights is made.

Excluded inputs include private continuity, raw MeshBoard conversation bodies, session logs, private recordings, dependency caches, commercial games, firmware and unrelated projects. The bookmaking thread supplied process guidance; it is not republished here. The other book's manuscript and cover were not imported.

The original cover vector, teaching additions and synthetic calculations were authored for this local edition. No publisher branding or affiliation is used. This register is a source-selection receipt, not legal clearance to publish the exact finished artifact.

Source register and edition boundary

This edition explains application commit `ab912d51ecc615f7aa55a4d27f6a876673739cdf`. The table identifies actual mechanisms, not just filenames. Links are pinned so later documentation-only commits do not silently change the cited source.

ID	Source and symbols	What this supports
S-01	`src/app.js`: `launch`, `loadRom`, `togglePower`, `rememberDevice`, fullscreen handlers	Host ownership, cartridge switching, distinct power/checkpoint behavior, same-session presentation
S-02	`src/player.js`: `initialize`, `configure`, `input`, `restore`, message listener	Pinned adapter, loader configuration, input sources, native VSync, preserved framebuffer and command protocol
S-03	`src/cartridge.js`: `identifyRom`, `latestSlot`, `encodeBackup`, `decodeBackup`	Content identity, size/header limits, runtime compatibility, bounded binary encoding and corruption detection
S-04	`src/save-store.js`: `SaveStore`, `putSlot`, `putBattery`, `#transaction`	Database stores, atomic updates, commit acknowledgement and explicit storage errors
S-05	`src/handheld.js`: `createHandheld`, extruded shapes and bezel texture	Original meshes, apertures, control relief and shared monochrome wordmark
S-06	`src/scene.js`: `updateCameraTargets`, `project`, `render`, `insertCartridge`	Projection, safe regions, animation, adaptive scene painting and view changes
S-07	`src/lighting.js`: `ScreenIllumination`, `createDeskLamp`, `createRoomLighting`	Linear-light averaging, sampling cadence, independent lights and approximated bounce
S-08	`src/keyboard.js`: `bindKeyboard`; `src/preferences.js`: `CONTROLS`, validators	Held-key cleanup, duplicate binding rejection, nine CRT choices and managed options
S-09	`src/startup.js`: `drawStartup`, `playStartup`, `playStartupSound`	High-resolution original boot art, sliced wave, reduced-motion behavior and synthesized chime
S-10	`scripts/site-files.mjs`, `scripts/build.mjs`	Static output allowlist, local vendor mapping and original demo generation

S-11	`tests/browser/dock.spec.mjs`, `tests/browser/scene.spec.mjs`	Native clock evidence, fullscreen, physical geometry, notice clearance and actual pinch regression
S-12	`.github/workflows/pages.yml`	Full build/browser gates before Pages deployment
S-13	`scripts/capture-readme.mjs`, `scripts/render-readme.py`	Actual browser recording, timing receipts, GIF encoding and separate banner generation

Later lettering delta

Workstream45's metallic chroma follow-up is a deliberate update beyond the pinned first edition above. Qualified `createHandheld` paints static chrome bands/tint inside the retained letter alpha; `app.js` shares that exact canvas with the fullscreen DOM. The verification record identifies qualification and delivery. The original source links and historical monochrome screenshots are not silently relabeled as this later edition.

Later rear-logo delta

Workstream46adds a larger colored engraving beyond the pinned first edition. Qualified `alphaContours` and `engraveRearLogo` reuse contour extraction, classify nested plastic islands and retain the original bright artwork on real inset floors. The verification record separates local qualification, actual delivery and aesthetic acceptance. Historical rear-view figures keep their original provenance.

Workstream 51 further enlarges the frame to 3.45 units and deepens the same colored cavities to .024 units without changing the artwork or cover. Qualified `engraveRearLogo` retains the same contour/floor/wall mechanism. Its separate verification record tracks this later parameter change; the original edition and 46 source stay pinned.

Later website identity delta

Workstream47separates chrome website identity (`index.html`, `assets/website-logo.png`, header shadows in `styles.css`) from the unchanged purple console loader in `src/brand.js`. Its verification record tracks local checks and public delivery separately. Additional consoles and the owner's repository rename are not implemented by that branding change.

Current documentation and media

The earlier workstream 48 refreshes README media on the actual public application, whose 29 assets independently match the qualified `f5bc639` source above. The recording checkout `9140b53` is documentation-only. That edition's `readme-*` figures and GIFs show the later chrome website,

purple console, metallic bezel and etched rear; the first-edition source pins and historical geometry figures remain intact. See Media for actual origin, timestamps, review scope and the previous local media edition.

Workstream 68 replaces shared README captures with the current local application source `82834df242aa7f0cec6cfa634e0966e7a3a500ec`. Figure B-01 and the shared startup GIF now use that capture edition; the book's pinned implementation sources and historical geometry figures remain unchanged. The new media receipt separates local footage, authorized GIF-only game content and publication.

Workstreams 69/70 subsequently refresh Figure B-01, five shared stills and pickup/inspection footage from local application `6fb1890`, and add an actual movable-dock GIF. No ROM is loaded in that take. See the dock/inspection receipt. The pinned book implementation edition remains unchanged.

Evidence is narrower than explanation

Current release records live in Verification; capture origins and historical figures live in Media. Those records distinguish local, software-rendered, hosted and actual public observations. The chapters explain why the mechanisms matter; they do not reclassify historical tests as fresh executions.

The original Pixel Lab program is the public demonstration cartridge. No commercial ROM, private continuity store or raw conversation is included. Third-party runtime and artwork obligations remain in Credits and licenses.